

Machine Learning per l'Ingegneria del Software

Gianpaolo Pestilli – 0386492

Indice

Introduzione	2
Metodologie adottate	2
<i>Milestone 1</i>	<i>2</i>
<i>Milestone 2</i>	<i>6</i>
<i>Milestone 2 – Esperimento personale.....</i>	<i>7</i>
<i>Milestone 3</i>	<i>7</i>
<i>Milestone 4</i>	<i>8</i>
<i>Milestone 4 – Esperimento personale.....</i>	<i>8</i>
Risultati ottenuti	9
<i>Milestone 1</i>	<i>9</i>
<i>Milestone 2</i>	<i>10</i>
<i>Milestone 2 – Esperimento personale.....</i>	<i>12</i>
<i>Milestone 3</i>	<i>13</i>
<i>Milestone 4</i>	<i>15</i>
<i>Milestone 4 – Esperimento personale.....</i>	<i>17</i>
Minacce alla validità.....	18
<i>Milestone 1</i>	<i>18</i>
<i>Milestone 2</i>	<i>19</i>
<i>Milestone 2 – Esperimento personale.....</i>	<i>20</i>
<i>Milestone 3</i>	<i>20</i>
<i>Milestone 4</i>	<i>21</i>
<i>Milestone 4 – esperimento personale</i>	<i>21</i>
Conclusioni	22
Riferimenti	22
<i>Link GitHub</i>	<i>22</i>
<i>Link SonarCloud.....</i>	<i>22</i>
Appendice	22

Introduzione

Con il presente documento si intende analizzare come la bugginess impatti l'Ingegneria del Software. Il progetto esaminato è "Apache Syncope". L'idea alla base dell'elaborato risiede nel fatto che la difettosità di una classe possa essere predicibile esaminando determinate feature: impiegare modelli di machine learning per tale scopo comporta un enorme vantaggio per prevenire difetti o concentrare meglio le risorse di test su artefatti software ritenuti a maggior rischio. Lo studio si suddivide in quattro milestone, ciascuna riguardante una specifica fase:

- Milestone 1 – recupero dei dati;
- Milestone 2 – addestramento sui dati estratti;
- Milestone 3 – analisi "what-if" per capire quanti difetti si sarebbero evitati se il codice fosse stato esente da smell;
- Milestone 4 – refactoring delle classi ad opera di un LLM, al fine di capire come cambia il codice generato in funzione degli input forniti.

Si precisa inoltre che sono state intraprese anche alcune iniziative personali (durante la Milestone 2 e la Milestone 4) ed esaminati anche scenari differenti rispetto a quelli strettamente legati al progetto: i risultati sono tutti documentati nei paragrafi successivi.

Metodologie adottate

Milestone 1

Nella Milestone 1 ci si occupa della creazione del dataset per il classificatore. L'attività è stata svolta seguendo questi step.

Prima di tutto è stato necessario reperire tutte le release disponibili del progetto "Apache Syncope". Dopodiché, sono stati confrontati i tag di Git presenti su GitHub con le release ID reperibili su Jira: sono state mantenute in analisi solo quelle release che risultavano avere sia un tag che un ID, così da essere certi di poter ricavare l'esatta istanza del codice da GitHub osservando i ticket Jira. Purtroppo, tale operazione non si è rivelata sufficiente. Un'analisi approfondita del progetto ha rivelato che gli sviluppatori della "Apache Foundation" sono soliti rilasciare delle release intermedie chiamate "Milestone" (contrassegnate dal suffisso -M). Queste sono versioni del codice volutamente instabili e piene di bug, aventi come unica finalità quella di essere poste sotto stress simulando esecuzioni reali per individuare difetti. Si è deciso di escludere queste release per evitare di contaminare il dataset con picchi anomali di classi buggy iniettate e corrette nel giro di una singola release. L'inclusione di queste versioni avrebbe infatti falsato il calcolo del Proportion: ticket aperti e chiusi quasi istantaneamente durante gli stress test ne avrebbero abbassato drasticamente il valore medio. Questa scelta metodologica ha comportato la necessità di ricollocare storicamente i commit: tutti i commit appartenenti alle release -M escluse non potevano essere ignorati poiché testimoniano che le classi sono state toccate, aumentandone il churn e la "temperatura interna" (fattori critici per la bugginess). I commit relativi alle release escluse sono stati pertanto considerati come facenti parte della prima release valida immediatamente antecedente a quella rimossa. Infine, si è riscontrata la presenza di numerose release rilasciate lo stesso giorno. Poiché da un commit si può risalire unicamente alla data, risultava impossibile determinare a quale esatta release esso afferisse, con il rischio di generare istanze duplicate e prive di dati aggiornati nel dataset. Per risolvere l'anomalia, tutte le release

rilasciate nel medesimo giorno sono state raggruppate nella release avente il tag più basso in ordine lessicografico.

Per mitigare il fenomeno dello snoring (ovvero il bias per il quale le release più recenti risultano avere sottostime sui difetti poiché causati da bug non ancora individuati) l'estrazione delle classi e la valutazione dei ticket si sono concentrate esclusivamente sul primo 33% delle release valide disponibili.

Sebbene il nome formale di una classe Java sia costituito dal suo package concatenato al nome del file .java, per stabilire l'identità e tracciare l'evoluzione di due classi nel tempo ci si è riferiti esclusivamente al nome del singolo file sorgente. Questa scelta si è resa necessaria in quanto, nelle prime release del progetto, le classi cambiavano frequentemente package e non sarebbero state correttamente tracciate ed etichettate mantenendo il path assoluto. Sono state considerati solo file .java che non fossero relativi a test.

Per ogni classe identificata nel 33% delle release, si è proceduto al calcolo delle feature attraverso due fonti:

- **SonarCloud (Metriche Statiche):** analizzando lo snapshot di ogni release, sono stati calcolati i valori di *LOC*, *numSmells* e *numOps*.
- **Git (Metriche di Processo):** Tramite la history dei commit, sono state calcolate le feature di tipo “in-window” (sulla singola release) e “FromBegin” (cumulate dal primo commit in assoluto). Tra queste figurano: *numRevisions*, *numFixes*, *numAuthors*, *churn*, le medie e i massimi di *LOCAdded* e *ChangeSet*, il tempo medio tra i commit (*avgTimeBetweenCommits*) e l'età della classe (*age*). È stata calcolata anche la *weightedAge*, pesata in base al *churn* dei singoli commit.

Una volta raccolti questi dati, è stato istanziato un dataset temporaneo in cui la label di classificazione (*isBuggy*) di tutte le classi è stata inizializzata a False.

Feature estratte

Metrica	Definizione
LOC	Numero totale di righe di codice presenti nel file sorgente, commenti inclusi.
numSmells	Numero complessivo di violazioni (code smell) rilevate sulla classe analizzando i sorgenti.
numOps	Numero totale di metodi definiti all'interno della classe.
numRevisions / numRevisionsFromBegin	Numero di commit totali che hanno modificato la classe.

Metrica	Definizione
$\frac{\text{numFixes}}{\text{numFixesFromBegin}}$	Numero di commit etichettati come correttivi (fix) che hanno toccato la classe.
$\frac{\text{numAuthors}}{\text{numAuthorsFromBegin}}$	Numero di autori distinti dei commit che hanno modificato la classe.
$\text{churn} / \text{churnFromBegin}$	Somma totale delle righe di codice aggiunte e rimosse per ciascun commit.
$\frac{\text{maxLOCAdded}}{\text{maxLOCAddedFromBegin}}$	Il numero massimo di righe aggiunte in un singolo commit osservato.
$\frac{\text{avgLOCAdded}}{\text{avgLOCAddedFromBegin}}$	Media delle righe aggiunte per commit, calcolata come righe aggiunte totali divise per il numero di revisioni.
$\frac{\text{maxChangeSet}}{\text{maxChangeSetFromBegin}}$	Massima dimensione del change set (file modificati insieme alla classe) osservata nei commit.
$\frac{\text{avgChangeSet}}{\text{avgChangeSetFromBegin}}$	Media della dimensione dei change set, calcolata come file totali modificati nei vari commit diviso il numero di revisioni.
$\text{avgTimeBetweenCommits}$	Tempo medio intercorso (in giorni) tra commit successivi che toccano la classe.
age	Età temporale della classe, calcolata in settimane a partire dal primo rilascio.
weightedAge	Età media ponderata della classe in settimane, pesata per il churn generato nei rispettivi commit.

Oltre alle feature classiche della letteratura di riferimento, sono state concepite e introdotte due metriche specifiche ideate ad hoc per cogliere ulteriori livelli di complessità del progetto: *numOps* e *avgTimeBetweenCommits*. Queste due metriche sono state reputate valide in quanto descrivono due vettori di instabilità differenti:

- ***numOps***: descrive la complessità strutturale e architetturale. Un numero elevato di metodi all'interno di una singola classe può denotare una violazione del Single Responsibility Principle aumentando la difficoltà di comprensione e manutenzione da parte degli sviluppatori.

- **avgTimeBetweenCommits**: descrive la "frequenza di perturbazione" del codice. Una media molto bassa indica commit ravvicinati nel tempo, sintomo frequente di instabilità, tentativi di risoluzione di bug iterativi o conflitti di sviluppo continui, fattori che impattano positivamente sulla probabilità che la classe risulti difettosa.

L'ultimo passaggio è consistito nell'individuazione dei difetti per assegnare le label corrette. Tramite Jira sono stati recuperati i ticket appartenenti al progetto di tipo "Bug" e con stato "Resolved" o "Closed". Per ciascun ticket si è proceduto alla determinazione temporale:

- **Opening Version (OV)**: Se un ticket risultava aperto tra due release X e Y, è stato posto $OV = X$ (ovvero la prima release precedente o uguale alla data di apertura).
- **Fixed Version (FV)**: Se un ticket risultava chiuso tra due release X e Y, è stato posto $FV = Y$.

Sono stati individuati ticket anomali aventi $OV = FV$. Tali ticket sono stati aperti dai programmatori esclusivamente per ragioni di tracciabilità, dopo aver già risolto il bug. Essendo la OV inconsistente, a maggior ragione l'elenco delle eventuali Affected Version (AV) non poteva essere considerato attendibile. Anche questi ticket inaffidabili la Injected Version è stata ricavata grazie a Proportion.

Per calcolare la Injected Version (IV) nei ticket coerenti, si è verificata la presenza delle Affected Version (AV). Se la prima AV riportata dal ticket risultava $\leq OV$, si è posto direttamente $IV = AV$. Nei ticket sprovvisti di AV affidabili, la IV è stata stimata utilizzando Proportion Total. Dopo aver calcolato il moltiplicatore medio P dai ticket completi ($P = \frac{FV-IV}{FV-OV}$), si è stimata la nuova Injected Version con la formula:

$$IV = FV - P \cdot (FV - OV)$$

A seguito del calcolo, tutti i ticket che risultavano avere $IV = FV$ (ovvero relativi a bug corretti nello stesso istante in cui sono stati iniettati) sono stati scartati in quanto non rappresentativi di difetti presenti nel codice.

Infine, applicando l'algoritmo SZZ ai commit di fix, è stato delineato il ciclo di vita di ciascun bug e si è proceduto al labeling. Per effettuare questa operazione è stato utilizzato il 100% delle release collezionate e non solo il 33% impiegato per l'estrazione di classi e feature. Questo approccio si rende necessario perché i bug introdotti nelle prime release vengono spesso scoperti e risolti solo in release molto successive. Ignorare lo storico futuro del progetto comporterebbe una mancata individuazione dei difetti latenti. Per ogni ticket validato, sono stati identificati i commit di risoluzione analizzando i log per trovare l'ID del ticket stesso e si sono individuate tutte le classi toccate da tali commit. La logica alla base di questo labeling è chiara: se una classe necessita di essere modificata per risolvere un bug segnalato, si assume logicamente che l'errore risiedesse al suo interno, rendendola la portatrice attiva del difetto. Di conseguenza, la difettosità si estende in modo retroattivo: la classe viene considerata "buggy" a partire dalla Injected Version (il momento in cui il bug è stato originariamente introdotto) fino alla Fixed Version esclusa (il momento della sua correzione).

Tutte le classi del dataset temporaneo che risultano modificate da un commit di fix hanno visto la propria label commutata in *isBuggy* = *True* per tutte le release comprese tra la IV e la FV esclusa, previa verifica che la classe esistesse effettivamente in tali versioni. Al contrario, le classi che non sono mai state toccate da alcun commit correttivo legato a un ticket hanno semplicemente mantenuto il loro stato originario, preservando l'attributo *isBuggy* = *False* per tutte le loro occorrenze.

Milestone 2

Arrivati alla Milestone 2, si riceve in input dalle fasi precedenti il dataset contenente la lista di tutte le classi Java estratte, le relative metriche e la label di bugginess. In questa fase il dataset viene pre-processato e utilizzato per addestrare e validare diversi modelli di Machine Learning. Tutte le operazioni svolte in questa milestone sono state automatizzate sviluppando un modulo Java custom basato sulle API della libreria Weka. Per gestire l'elevato carico computazionale delle molteplici configurazioni, l'esecuzione è stata parallelizzata implementando un pool di thread, demandando a ciascuno di loro il completamento di un esperimento indipendente.

Invece di ricorrere a una banale standardizzazione, i dati sono stati normalizzati in modo mirato applicando il logaritmo in base 10 (tramite la funzione matematica $\log(1 + x)$) agli attributi numerici caratterizzati da maggiore variabilità, al fine di smussare i picchi e rendere le distribuzioni più trattabili dai modelli. I valori risultati intrinsecamente stabili nel tempo non hanno necessitato di tale normalizzazione e sono stati mantenuti nel loro formato originale (nello specifico: *ReleaseID*, *numAuthorsFromBegin*, *avgChangeSetFromBegin*, *maxChangeSetFromBegin* e *age*).

Per individuare il modello predittivo migliore, il dataset processato è stato utilizzato per valutare tre diversi classificatori: *Random Forest*, *Naive Bayes* e *IBk*. Le prestazioni di ciascun modello sono state misurate confrontando le metriche di: *Precision*, *Recall*, *Area Under ROC (AUC)*, *Kappa* e *Accuracy*. Sono stati inoltre estratti i valori assoluti della matrice di confusione (*TP*, *FP*, *TN*, *FN*).

I modelli sono stati testati impiegando due diverse tecniche di validazione:

- ***Ten Times Ten Folds Cross Validation***: tecnica standard che garantisce robustezza statistica eseguendo molteplici partizionamenti del dataset, variando casualmente il seed.
- ***Ordered Holdout (Split 80-20)***: tecnica introdotta per ovviare a un grave limite della Cross Validation, la quale mescolando le istanze viola inevitabilmente l'ordine cronologico delle release. L'Ordered Holdout simula uno scenario predittivo temporalmente coerente, addestrando il modello sull'80% più vecchio del dataset e testandolo sul 20% più recente.

Ogni classificatore è stato testato attraverso diverse combinazioni per valutare come le tecniche di campionamento e riduzione della dimensionalità influenzino le predizioni:

- ***Feature Selection***: all'atto pratico, la selezione delle feature serve a ridurre la dimensionalità del dataset, scartando attributi non informativi o fortemente correlati tra loro. Questo riduce il rischio di overfitting e accelera l'addestramento, fornendo al classificatore solo le metriche che presentano una reale capacità predittiva rispetto alla bugginess. A livello software, è stata applicata la tecnica *Filter* utilizzando il valutatore *weka.attributeSelection.CfsSubsetEval* (che valuta il potere predittivo dei sottoinsiemi di attributi) abbinato all'euristica di ricerca *weka.attributeSelection.GreedyStepwise*. Per garantire un'esplorazione ottimale, la ricerca è stata impostata in modalità backward (*setSearchBackwards(true)*), corrispondente al flag -B, in modo da partire dall'insieme completo di metriche ed eliminare progressivamente quelle ridondanti.
- ***Balancing***: poiché i bug software sono eventi relativamente rari, il dataset presenta un naturale sbilanciamento verso le classi non difettose (classe maggioritaria). I classificatori addestrati su dati sbilanciati tendono a ignorare la classe minoritaria. Per forzare il modello a

riconoscere i pattern dei difetti, il dataset deve essere riequilibrato. Sono state testate due alternative:

- **Undersampling:** questa tecnica bilancia il dataset rimuovendo casualmente istanze dalla classe maggioritaria finché non eguaglia quella minoritaria. In Weka, è stata implementata tramite il filtro `weka.filters.supervised.instance.SpreadSubsample` configurando il parametro `DistributionSpread` a 1.0 (flag `-M 1.0`), per imporre esplicitamente una proporzione 50/50. È l'ideale per
- **SMOTE:** invece di rimuovere dati o duplicare banalmente le istanze esistenti, SMOTE genera sinteticamente nuovi esempi della classe minoritaria interpolando i valori tra i k-vicini più prossimi, limitando fortemente il rischio di overfitting. È stato implementato richiamando il filtro `weka.filters.supervised.instance.SMOTE` con i suoi parametri di default.

Per ogni esperimento la Feature Selection ha preceduto il Balancing per non alterare artificialmente l'importanza degli attributi originali.

Milestone 2 – Esperimento personale

In parallelo alle attività della Milestone 2, è stato condotto un esperimento esplorativo per verificare se le performance dei classificatori dipendessero effettivamente dalle metriche strutturali o se gli algoritmi stessero sfruttando identificativi testuali come scorciatoia per la predizione. A tal fine, è stata implementata una procedura di *'Manual Cut'* che ha rimosso dal dataset le colonne relative a *release ID*, *project name* e *class name*. L'obiettivo era forzare i modelli ad addestrarsi esclusivamente sulle altre metriche, eliminando variabili identificative potenzialmente “fuorvianti”. A livello implementativo, si è utilizzato il filtro `Weka weka.filters.unsupervised.attribute.Remove` attivando il flag `-R 1-3` (tramite `setAttributeIndices("1-3")`), rimuovendo così fisicamente dal dataset le colonne precedentemente citate.

Milestone 3

L'obiettivo della terza milestone è valutare l'impatto dei code smell sulla difettosità del software mediante un'analisi di tipo "What-If". Questa tecnica permette di simulare scenari ipotetici in cui i difetti legati alle cattive pratiche di programmazione (smell) vengono teoricamente risolti, stimando come l'assenza di tali anomalie influenzerebbe le predizioni del modello di machine learning.

A tal fine, partendo dal dataset completo precedentemente generato (definito Dataset A), sono stati isolati tre sotto-dataset specifici per l'esperimento:

- **Dataset B+:** un sottoinsieme del Dataset A contenente esclusivamente le istanze (classi) che presentano almeno un code smell.
- **Dataset B (Scenario What-If):** una copia esatta del Dataset B+, in cui però il valore della colonna relativa ai code smell è stato forzato artificialmente a 0. Questo dataset simula lo scenario ideale in cui tutte le classi affette da smell siano state sottoposte a un refactoring correttivo.

- **Dataset C:** un sottoinsieme del Dataset A composto unicamente dalle classi originariamente prive di code smell.

Il classificatore prescelto è un Random Forest configurato nel seguente modo:

- **Feature Selection (FILTER):** implementata per ridurre la dimensionalità e isolare le metriche maggiormente predittive, utilizzando la valutazione *CfsSubsetEval* con ricerca di tipo *backward (GreedyStepwise)*.
- **Balancing (SMOTE):** applicato per gestire lo sbilanciamento nativo del dataset, generando istanze sintetiche della classe minoritaria al fine di migliorare la sensibilità del modello.

Tale modello, addestrato sull'intero Dataset A, è stato inizialmente testato sui dataset "reali" (A, B+ e C) per verificarne la coerenza predittiva su porzioni di codice caratterizzate da diverse densità di smell. Successivamente, il classificatore è stato testato sul Dataset B (lo scenario What-If) al fine di verificare se, eliminando matematicamente il conteggio degli smell, il modello fosse portato a riclassificare come *non-buggy* le istanze precedentemente identificate come *buggy*.

Infine, per validare criticamente l'esito di questa simulazione, è stata eseguita un'analisi di correlazione tra le metriche strutturali, la variabile associata agli smell e la label defectiveness. Questo passaggio metodologico è fondamentale per determinare se l'informazione predittiva apportata dai code smell sia univoca, oppure se risulti assorbita e veicolata da altre feature architetturali fortemente correlate a essi.

Milestone 4

L'ultima fase del progetto è focalizzata sulla valutazione delle capacità dei Large Language Model (LLM) nell'eseguire operazioni di refactoring automatico del codice. L'obiettivo è azzerare i code smell preservando intatte le funzionalità originarie e la correttezza semantica validata dalle suite di test. L'esperimento è stato condotto sul modulo *core* del progetto "Apache Syncope". Sono state selezionate due classi con profili di complessità differenti per valutare l'impatto della dimensione del contesto sulle performance dell'LLM:

- **Classe A (DefaultMappingManager):** alta complessità, 40 code smell iniziali (2 Blocker, 15 High, 18 Medium, 5 Low).
- **Classe B (DefaultPasswordGenerator):** bassa complessità, 2 code smell iniziali (1 High, 1 Medium).

Per istruire l'LLM è stato formulato il seguente prompt di base, progettato per imporre vincoli rigidi sulla conservazione delle firme dei metodi della classe generata:

"You are an expert Java developer. I want to improve the maintainability of the attached class. Create a new version that preserves the exact same functionality and public behavior while removing the attached code smells. Make sure the new class passes the attached tests, which currently pass on the class and must continue to pass unchanged. Do not include any change that goes beyond what has been explicitly requested (i.e., only the smell-removal refactoring). The new

class must be a drop-in replacement for the original one: same public method signatures, same class name conventions where required, and the same interaction with the rest of the system. This is an important request: take all the time you need to analyze the code carefully and provide a complete, accurate, and correct answer."

A questo prompt sono stati affiancati input incrementali per generare 4 varianti della stessa classe (C_1 , C_2 , C_3 , C_4), fornendo progressivamente all'IA i test Black-Box, Control-Flow e Mutation Test.

Le classi prodotte sono state analizzate per rispondere a quattro quesiti fondamentali:

1. La classe C_X compila (da sola o con il sistema)?
2. La classe C_X presenta code smell (nessuno, vecchi o nuovi)?
3. Esiste una feature positivamente correlata con la difettosità che è maggiore in C_X rispetto a C_0 ?
4. Esiste una feature negativamente correlata con la difettosità che è maggiore in C_X rispetto a C_0 ?

L'LLM scelto è M365 Copilot (opzione "Think Deeper").

Milestone 4 – Esperimento personale

L'LLM ha più difficoltà a rifattorizzare la classe con il maggior numero di smell oppure quella con il tempo di refactoring più alto? I criteri di difficoltà di un essere umano sono gli stessi per un'IA? È stata considerata la classe con più smell in assoluto dell'intero progetto e la classe con il tempo di refactoring più elevato. Tali file sono rispettivamente *ClientExceptionType.java* (110 righe di codice, 55 smell ma 113 minuti per essere rifattorizzata) e *TopologyTogglePanel.java* (691 righe di codice, 23 smell ma 6422 minuti per essere rifattorizzata). Già da questa prima analisi si intuisce il valido motivo della domanda: può una ENUM essere più complessa di una classe Java completa? Per capirlo è stato chiesto a M365 Copilot (opzione "Think Deeper") di rifattorizzare entrambe le classi utilizzando lo stesso prompt della Milestone 4: ovviamente non erano disponibili test, quindi è stata generata solo la variante C1 per entrambe le classi. Ci si chiede se le varianti compilano e poi si calcolerà il rapporto tra il numero di smell post refactoring e il numero di smell pre-refactoring.

Risultati ottenuti

Milestone 1

L'esecuzione della pipeline di estrazione ed elaborazione della Milestone 1 ha permesso di generare il dataset di base per il progetto "Apache Syncope" e ha fatto emergere dinamiche molto interessanti riguardo al ciclo di vita dei difetti e all'affidabilità dei sistemi di tracciamento.

L'analisi ha rivelato che complessivamente il 7,884% delle classi analizzate nel dataset si è rivelato essere difettoso (buggy). Questa percentuale, relativamente contenuta, suggerisce che il debito tecnico non è distribuito in modo uniforme nell'intero progetto, ma tende a concentrarsi in un

sottoinsieme specifico di classi critiche. Inoltre, questo valore evidenzia un marcato e fisiologico sbilanciamento del dataset (le classi non difettose superano il 92%).

Dal punto di vista della qualità dei dati estratti da Jira, l'analisi ha evidenziato diverse anomalie: si è riscontrato che il 3,4% dei ticket analizzati non aveva alcun commit risolutivo associato. Questo fenomeno è generalmente riconducibile a ticket aperti per discutere problemi di configurazione, task di documentazione o difetti che non hanno richiesto la modifica di alcun file. Inoltre, un marginale 0,26% dei ticket presentava l'inconsistenza temporale estrema di avere la Injected Version (IV) coincidente con la Fixed Version (FV). L'identificazione e lo scarto di questi bug "istantanei", spesso frutto di ticket aperti retroattivamente per mera tracciabilità amministrativa, hanno permesso di ripulire il dataset da un notevole rumore statistico.

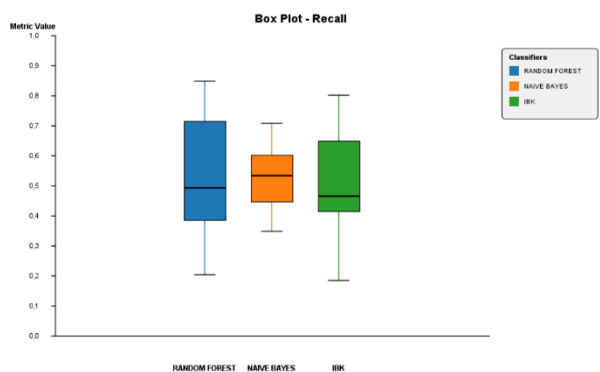
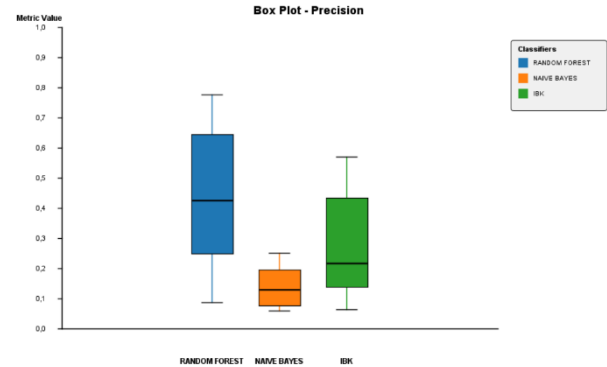
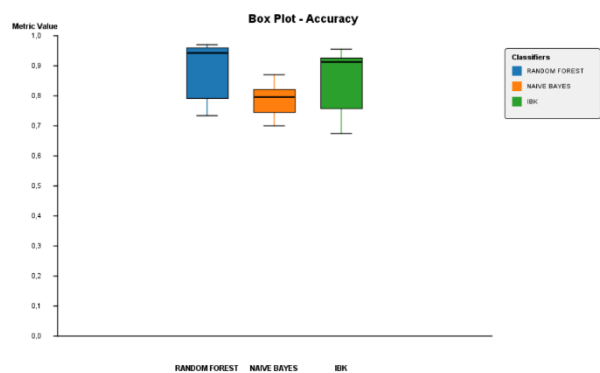
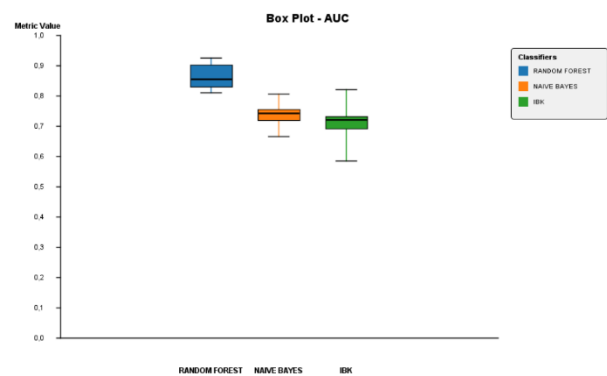
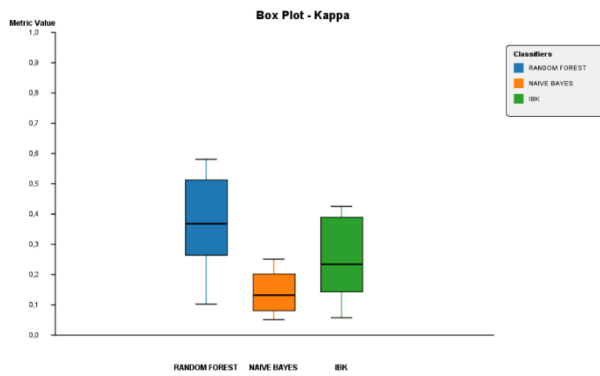
Particolarmente rilevante è stato l'impatto del calcolo della Injected Version. È emerso che per ben il 21,82% dei ticket presi in esame non era stata compilata correttamente alcuna Affected Version (AV) su Jira. Questo dato sottolinea come i developer non siano sempre diligenti nella compilazione dei metadati, rendendo l'uso dell'euristica Proportion strettamente necessaria.

L'applicazione di Proportion Total ha restituito un moltiplicatore medio P pari a 1,284. Il fatto che P sia maggiore di 1 è significativo: stando alla formula $IV = FV - P \cdot (FV - OV)$, un moltiplicatore $P > 1$ indica che la Injected Version stimata tende a collocarsi storicamente prima della Opening Version (OV). Questo conferma la dinamica per cui, nella stragrande maggioranza dei casi, il bug viene iniettato nel sistema svariate release prima che venga individuato e segnalato tramite ticket.

Inoltre, questo specifico valore di P convalida a posteriori l'assunzione metodologica iniziale di scartare le release "Milestone" (-M). Nelle versioni più recenti di "Apache Syncope", infatti, queste release di test tendono ad alternarsi in modo sempre più fitto e regolare alle release ufficiali. Se non fossero state filtrate, l'inclusione di queste innumerevoli versioni intermedie avrebbe "dilatato" artificialmente le distanze tra le versioni, portando il moltiplicatore Proportion verso un valore prossimo a 2. Un $P \approx 2$ avrebbe comportato una sovrastima eccessiva del ciclo di vita dei bug, retrodatando le Injected Version in modo irrealistico e falsando in modo irreparabile il labeling delle classi nel dataset temporaneo.

Milestone 2

In questa sezione si presentano i risultati ottenuti dall'applicazione dei modelli di Machine Learning. L'analisi è finalizzata a identificare la configurazione che garantisce le migliori performance nella predizione delle classi *buggy*. all'analisi delle metriche aggregate, emerge una chiara superiorità del classificatore **Random Forest**: è il modello più performante in quasi tutte le metriche (*Accuracy*, *AUC*, *Kappa*), tranne che in *Precision* dove ha alta variabilità. Naive Bayes è il più "prevedibile" (box sempre stretti) ma spesso con valori più bassi, specialmente in *Precision* e *Kappa*. **IBk** è il più incostante: box spesso larghi con whisker lunghi, soprattutto in *Accuracy* e *Precision*. *Recall* è la metrica con più incertezza per tutti: whisker lunghi ovunque, segno che nessun modello è particolarmente stabile su questa metrica. Il gap tra *Precision* e *Recall*, specialmente per **Naive Bayes**, suggerisce che questo modello probabilmente predice molti falsi positivi.



I grafici sopra riportati si riferiscono alle performance medie di tutti i classificatori in tutti gli scenari riportati. In appendice, invece, si potranno consultare i valori delle metriche per ogni scenario con le relative matrici di confusione. Di seguito, infatti, si illustrano solo i grafici relativi agli esperimenti più significativi.

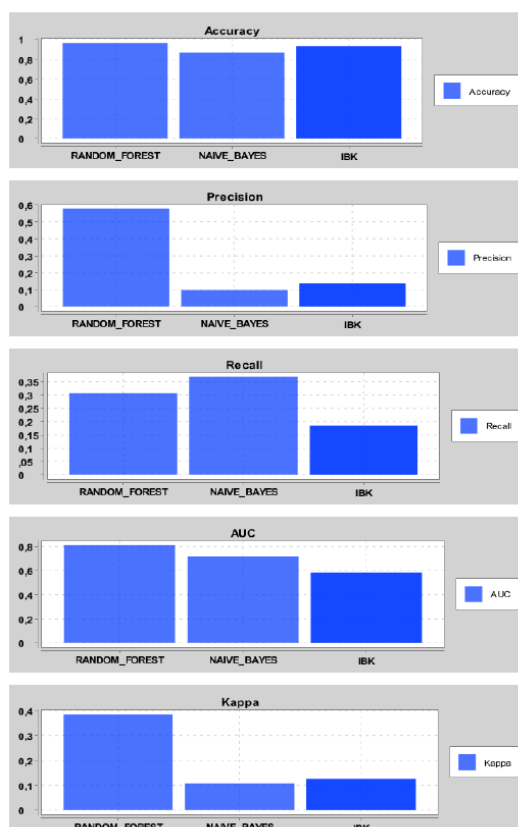
Il ranking del classificatore cambia in base alla metrica considerata.

Per **Accuracy**, **Precision**, **AUC**, **Kappa**: Random Forest è quasi sempre primo.

Su **Recall**: Naive Bayes o IBk raggiungono valori pari o superiori a Random Forest, pertanto la classifica quindi si inverte proprio su questa metrica.

Metrica	Il migliore	Note
Accuracy	Random Forest	Quasi sempre primo, IBk talvolta vicino
Precision	Random Forest	Nettamente il migliore in ogni configurazione

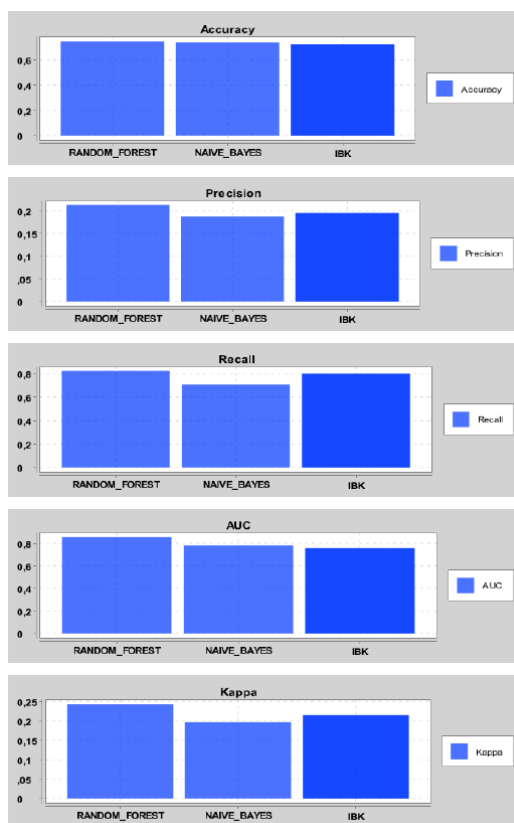
Recall	Naive Bayes (o IBk)	Soprattutto con undersampling, a scapito della Precision
AUC	Random Forest	Il gap è il più marcato tra tutte le metriche
Kappa	Random Forest	Conferma il miglior accordo complessivo tra predizione e realtà



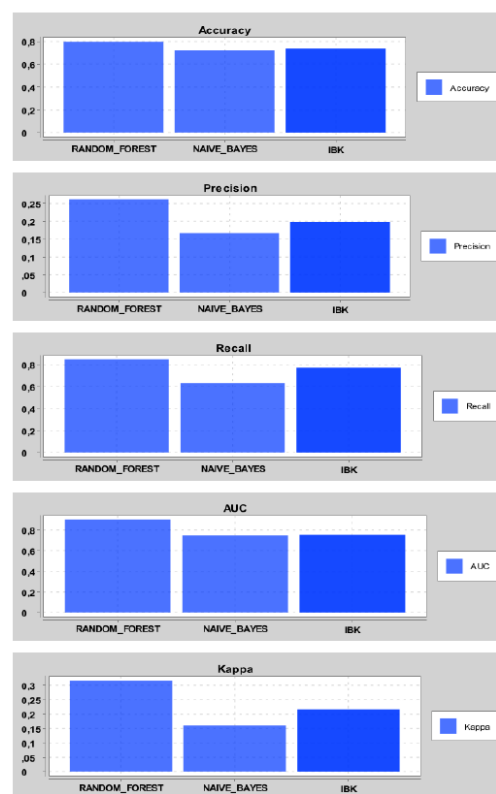
Manual Cut: YES; Feature Selection: FILTER; Balancing: NO; Validation: OH

Milestone 2 – Esperimento personale

In questa sezione ci si interroga sull'applicazione del "Manual Cut": rimuovere le colonne relative al nome del progetto, della release e della classe ha aiutato il classificatore? Random Forest è il classificatore che ci guadagna di più, ma solo se si effettua Undersampling con una Ten Times Ten Fold Cross Validation. Se si opta per un Ordered Holdout si osserva un aumento di *Precision* ma un decremento di *Recall*. Naive Bayes e IBk tendono invece a non migliorare (se non a peggiorare) in quasi tutte le metriche quando viene applicato il cut.



Manual Cut: NO; Feature Selection: FILTER;
Balancing: UNDERSAMPLING; Validation: TTTCV



Manual Cut: YES; Feature Selection: FILTER;
Balancing: UNDERSAMPLING; Validation: TTTCV

Milestone 3

La prima fase di analisi ha misurato l'accuratezza del classificatore sui dataset reali inalterati (Dataset A, B+ e C).

Dataset	Actual	Expected
A	1179	1173
B+	639	3706
B	-	3960
C	540	537

Tabella riassuntiva dei valori reali (Actual) e predetti (Expected) per i dataset di valutazione

Osservando il Dataset A (l'intero ecosistema), il modello risulta estremamente accurato: stima 1173 classi defective a fronte delle 1179 reali. Tuttavia, isolando il Dataset B+ (classi con code smell), si osserva una massiccia sovrastima predittiva (3706 classi stimate contro 639 reali). Questo fenomeno è un effetto collaterale indotto dall'utilizzo di SMOTE. L'algoritmo di bilanciamento ha reso il classificatore ipersensibile alla classe minoritaria (*buggy*). Di conseguenza, esposto a un sotto-dataset

intrinsecamente composto da classi complesse e con cattive pratiche di design, il modello classifica facilmente come *buggy*, generando un elevato volume di falsi positivi. L'accuratezza si mantiene quindi elevata sul panorama generale (A), ma tende al conservativismo estremo (segnalando tutto come potenziale difetto) sulle classi strutturalmente più pesanti.

Forzando artificialmente il conteggio degli smell a zero (Dataset B), si è quantificato il numero di classi *buggy* teoricamente prevenibili. In base alle predizioni del modello, il numero di classi *buggy expected* passa dalle 3706 del Dataset B+ alle 3960 del Dataset B.

Pertanto:

- L'azzeramento degli smell non ha ridotto le predizioni di difettosità, registrando anzi un paradossale lieve incremento.
- Nessuna porzione di difettosità è risultata direttamente attribuibile alla singola variabile quantitativa dei code smell.
- La manipolazione isolata degli smell senza alterare la struttura del codice non previene alcun difetto agli "occhi" del classificatore.

La motivazione per cui il classificatore mantiene inalterata la predizione di difettosità nello scenario What-If è spiegabile attraverso l'analisi della correlazione tra le feature.

Variable	Mean A	Mean B+	Mean B	Mean C	Corr. NSmells	Corr. Defectiveness
LOC	126.89	213.99	213.99	91.25	0.40*	0.26*
numRevisions	1.02	1.60	1.60	0.79	0.08*	0.06*
numRevisionsFromBegin	6.67	10.85	10.85	4.96	0.23*	0.19*
numFixes	0.28	0.47	0.47	0.21	0.12*	0.14*
numFixesFromBegin	2.63	4.49	4.49	1.87	0.22*	0.17*
numAuthors	0.49	0.67	0.67	0.41	0.10*	0.11*
numAuthorsFromBegin	2.31	3.22	3.22	1.94	0.22*	0.19*
churn	43.05	79.97	79.97	27.94	0.08*	0.04*
churnFromBegin	293.21	525.49	525.49	198.18	0.31*	0.21*
maxLOCAdded	20.56	34.79	34.79	14.74	0.11*	0.06*
maxLOCAddedFromBegin	112.79	198.20	198.20	77.85	0.37*	0.27*
avgLOCAdded	12.31	18.71	18.71	9.69	0.08*	0.06*
avgLOCAddedFromBegin	71.89	116.81	116.81	53.52	0.35*	0.17*
avgChangeSet	192.28	192.51	192.51	192.19	-0.01	-0.03*
avgChangeSetFromBegin	995.37	1029.03	1029.03	981.60	-0.00	-0.06*
maxChangeSet	276.23	302.41	302.41	265.52	0.01	-0.02*
maxChangeSetFromBegin	1242.72	1349.53	1349.53	1199.03	0.04*	0.01
age	184.35	184.54	184.54	184.27	-0.00	-0.08*
weightedAge	61.94	78.93	78.93	55.00	0.08*	0.08*
numOps	6.33	7.21	7.21	5.96	0.06*	0.14*
avgTimeBetweenCommits	0.29	0.34	0.34	0.27	0.03*	0.04*
numSmells	1.08	3.73	0.00	0.00	-	0.10*

Tabella delle correlazioni tra le feature strutturali, i code smell e la difettosità

La tabella evidenzia come la presenza di code smell sia positivamente correlata con le metriche dimensionali e storiche della classe. Nello specifico, si osservano correlazioni statisticamente significative per le seguenti metriche, le quali risultano a loro volta correlate con la variabile *defectiveness*:

- **LOC:** correlazione di 0.40 con gli smell e 0.26 con la difettosità.
- **maxLOCAddedFromBegin:** correlazione di 0.37 con gli smell e 0.27 con la difettosità.
- **avgLOCAddedFromBegin:** correlazione di 0.35 con gli smell e 0.17 con la difettosità.
- **churnFromBegin:** correlazione di 0.31 con gli smell e 0.21 con la difettosità.

Tali valori dimostrano che l'informazione predittiva apportata dai code smell non è univoca, ma fortemente ridondante rispetto a queste metriche ingegneristiche. Il classificatore *Random Forest*, in fase di addestramento, ha appreso a mappare la difettosità sulla dimensione del file (LOC) e sulla mole di modifiche storiche (come il Churn o le righe di codice aggiunte nel tempo).

Nel Dataset B (What-If), il numero di smell viene forzato a zero, ma i valori di LOC e Churn rimangono quelli originati da un codice stratificato e complesso. Il modello, coerentemente, continua a segnalare il potenziale difetto leggendo l'immutata complessità strutturale. Ciò dimostra empiricamente che la mera rimozione dello smell, priva di un'effettiva semplificazione architetturale della classe, non ha alcun impatto risolutivo sulla predizione della difettosità.

Milestone 4

Le classi rifattorizzate dall'LLM sono:

A: DefaultMappingManager.java

B: DefaultPasswordGenerator.java

Di seguito i risultati dell'analisi basata sui quattro quesiti di valutazione.

Per quanto concerne la classe A, vale quanto segue.

La classe parte con un numero originale di smell pari a 40. I refactoring hanno dato origine alle varianti C1, C2, C3 e C4.

1. La classe C_x compila (da sola o con il sistema)? Tutte le varianti prodotte dal refactoring compilano correttamente sia in isolamento sia integrate con il resto del sistema Syncope.
2. La classe C_x presenta code smell (nessuno, vecchi o nuovi)? L'LLM non è riuscito ad azzerare gli smell a causa delle dimensioni della classe e dei vincoli del prompt: C1 scende a 27 smell, C2 a 18, C3 tocca il minimo di 15 e C4 risale a 16. Sono tutti smell ereditati dalla classe originaria, alcuni dei quali non rimovibili: gli smell relativi al troppo elevato numero di parametri presenti nel costruttore e in alcuni metodi non potevano essere risolti senza violare la direttiva del prompt (lasciare intatte le signature).

3. Esiste una feature positivamente correlata con la difettosità che è maggiore in C_X rispetto a C_0 ? La metrica LOC (Lines of Code), che risulta positivamente correlata alla difettosità, è aumentata in tutte le varianti prodotte. Nel tentativo di risolvere gli smell di complessità, l'LLM ha generato codice più prolisso, il che, da un punto di vista strettamente numerico, potrebbe indicare un mancato miglioramento della manutenibilità. Analizzando meglio la struttura interna del codice, si nota invece che le righe aggiunte sono state impiegate per la scrittura di metodi privati extra che potessero separare le responsabilità e scomporre la complessità dei metodi più corposi.
4. Esiste una feature negativamente correlata con la difettosità che è maggiore in C_X rispetto a C_0 ? Non si registrano incrementi significativi in feature storiche o metriche che indichino un effettivo abbassamento della difettosità strutturale.

Sotto il profilo semantico, la Classe A ha rappresentato un fallimento per l'LLM. Le varianti perdono la gestione originale della `NullPointerException`, causando il fallimento dei test strutturali (BB, CF e Mutation). Il problema principale risiede nella saturazione della context window: costretto a processare il codice della classe, l'elenco dei 40 smell e le lunghissime suite di test, l'LLM ha subito un degrado dell'attenzione, dimenticando porzioni di logica originaria pur di snellire il codice.

Per quanto riguarda la classe B, vale quanto segue.

La classe parte con un numero originale di smell pari a 2. I refactoring hanno dato origine alle varianti C1, C2, C3 e C4.

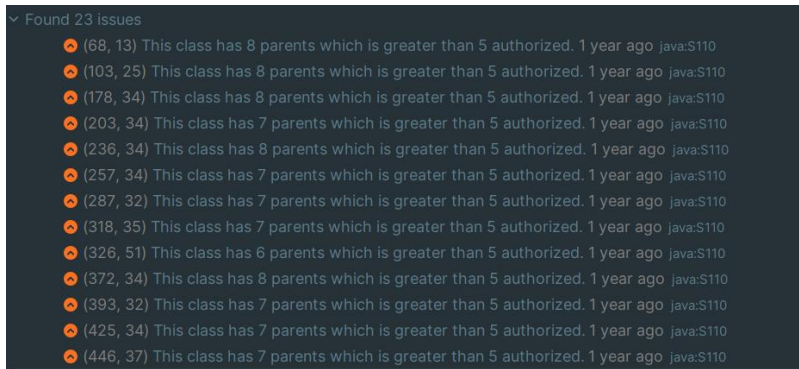
1. La classe C_X compila (da sola o con il sistema)? Tutte le varianti prodotte dal refactoring compilano correttamente sia in isolamento sia integrate con il resto del sistema Syncope.
2. La classe C_X presenta code smell (nessuno, vecchi o nuovi)? Dalla variante C1 in poi, i 2 code smell originari (1 High, 1 Medium) sono stati completamente eliminati.
3. Esiste una feature positivamente correlata con la difettosità che è maggiore in C_X rispetto a C_0 ? La metrica LOC (Lines of Code), che risulta positivamente correlata alla difettosità, è aumentata in tutte le varianti prodotte. Anche in questo caso l'LLM ha migliorato la manutenibilità, inserendo metodi che potessero separare meglio le responsabilità.
4. Esiste una feature negativamente correlata con la difettosità che è maggiore in C_X rispetto a C_0 ? No, le metriche non subiscono variazioni positive rilevanti.

Avendo una context window scarica grazie alle ridotte dimensioni della classe B, l'LLM ha effettuato un refactoring certosino. Tuttavia, ha introdotto allucinazioni semantiche non richieste. Per far passare alcuni test forniti nel prompt, l'IA ha inventato un metodo che intercetta i valori nulli lanciando una `IllegalArgumentException` al posto della originaria `NullPointerException`. Questo mascheramento semantico fa compilare la classe, ma fa fallire i test generati originariamente, dimostrando che l'LLM può alterare il contratto delle eccezioni in modo silente.

Milestone 4 – Esperimento personale

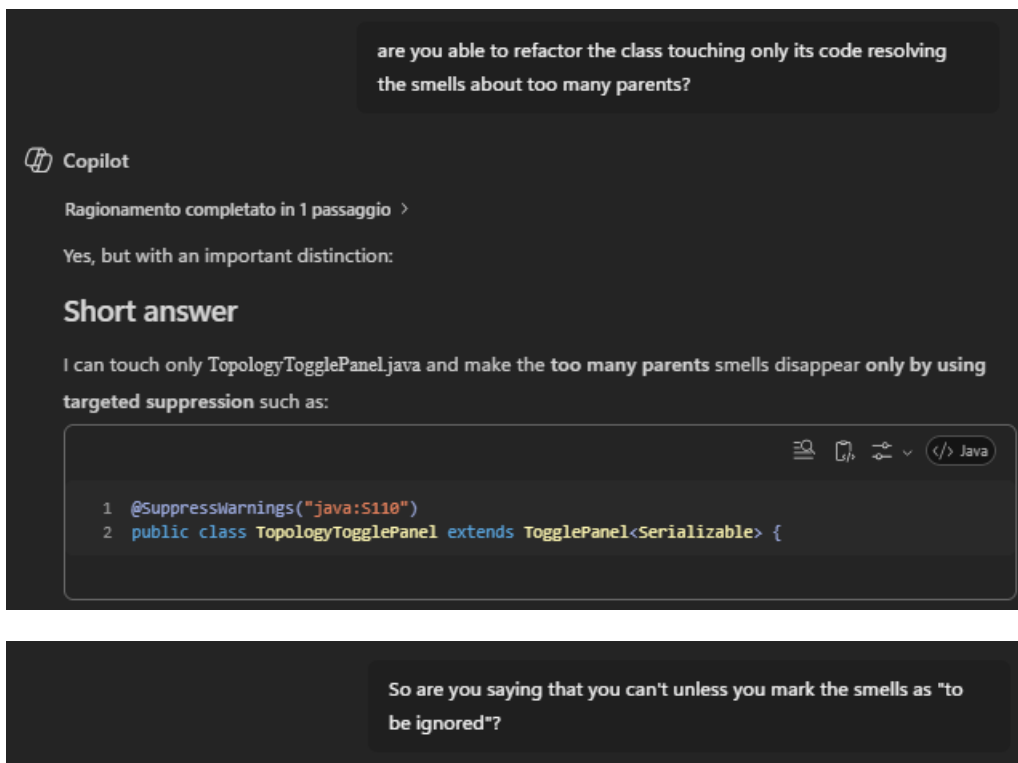
La classe *ClientExceptionType* aveva solo smell relativi al bad naming delle costanti della enum, i cui nomi non rispettavano le convenzioni Java. La classe rifattorizzata ha 0 smell e compila perfettamente. Il rapporto tra smell pre e post refactoring è dunque 0.

La classe riportava tutti smell di questo tipo.



Sono smell dovuti al fatto che la classe e quelle da essa invocate ereditano da troppe classi superordinate: l'albero di ereditarietà è troppo profondo.

Dopo una serie di prompt a seguito dei quali Copilot non ha mai fornito una implementazione valida, gli è stato chiesto apertamente se fosse capace di effettuare refactoring senza sopprimere gli smell manualmente. Di seguito gli screenshot estratti dalla conversazione.



So the honest answer is

Yes: if we preserve exact behavior, java:S110 can only be resolved by suppressing / ignoring it locally with justification.

For example:

```
1 @SuppressWarnings("java:S110")
2 public class TopologyTogglePanel extends TogglePanel<Serializable> {
```

or, for anonymous Wicket components:

```
1 new IndicatingAjaxLink<Void>("id") { // NOSONAR java:S110: wicket inheritance depth is
  framework-defined.
2   @Override
3   public void onClick(final AjaxRequestTarget target) {
4     ...
5   }
6 }
```

Bottom line

For java:S110: yes, suppression is the correct behavior-preserving solution.

Trying to “really” refactor the inheritance depth away would require changing framework inheritance or component types, which is no longer a smell-only refactoring and would risk breaking the Wicket UI behavior.

Dal momento che il prompt richiedente una classe con stessa segnatura, l’LLM si rifiuta di effettuare refactoring. Il rapporto tra smell post-refactoring e pre-refactoring è 1 perché la classe non è cambiata.

Si ritiene che il caso che si è palesato sia profondamente patologico, ma rappresenta un’ottima argomentazione a favore della tesi secondo cui il numero di smell non è ciò che più impatta sulla complessità di una classe dalla prospettiva di un LLM.

Minacce alla validità

Milestone 1

Aver adottato l'approccio Total (che calcola P sull'intero insieme dei ticket disponibili) si introduce in particolare un “lookahead bias”, in quanto si utilizzano informazioni del futuro per stimare il comportamento di ticket aperti nel passato: questo mette il classificatore in una condizione irrealistica, portando a una conseguente sovrastima delle sue reali capacità. Approcci alternativi, come Moving Window o Incremental, avrebbero fornito stime teoricamente più rigorose. Utilizzando tutti i ticket disponibili per calcolare P, inoltre, si corre il rischio che i primissimi ticket non siano più

rappresentativi della realtà attuale, poiché nel frattempo il sistema potrebbe essere cambiato profondamente.

Per quanto riguarda il calcolo delle metriche, vale quanto segue.

- ***Lines of Code (LOC)***: le righe di codice di una classe sono state valutate tenendo in considerazione anche i commenti. Questo vuol dire che il numero di LOC nel dataset potrebbe aumentare vertiginosamente da una release all'altra esclusivamente in base a quanto gli sviluppatori hanno commentato, senza aver per forza scritto del nuovo codice logico. La scelta è stata tuttavia obbligata: il calcolo del Churn e delle righe aggiunte è stato effettuato servendosi dei log di Git, che tengono intrinsecamente conto dei commenti, e si è preferito rendere le LOC coerenti con le restanti feature di processo.
- ***Gestione degli Smell e delle Milestone***: il numero di code smell è stato valutato calcolandolo dopo l'ultimo commit antecedente al rilascio ufficiale. Nonostante si sappia che gli smell di una release impattano pesantemente sulla successiva, non è stato possibile traslare i dati dalla release precedente a causa dei filtri effettuati sulle versioni -M. Si prenda l'esempio in cui, tra una ipotetica Release 10 e una Release 11, vi siano state due release di tipo Milestone uscite lo stesso giorno (entrambe condensate ed eliminate). A questo punto non avrebbe alcun senso affermare che siano stati gli smell di fine Release 10 ad impattare la Release 11, essendoci state di mezzo due evoluzioni del codice non visibili. Si potrebbe obiettare di importare per la Release 11 gli smell dell'ultima Milestone scartata, ma ciò genererebbe due problemi irrisolvibili: in primo luogo si importerebbe un numero di difetti spaventosamente alto e irrealistico (ereditato da una Milestone volutamente instabile) e in secondo luogo, poiché le due Milestone coincidevano come data, diverrebbe impossibile stabilire su quale branch di Git effettuare il checkout per SonarCloud (in caso di discrepanze tra le due, non si saprebbe quale dato utilizzare).
- ***Numero di autori distinti***: sono stati calcolati confrontando gli indirizzi e-mail di coloro che hanno committato. Se un autore cambia e-mail viene conteggiato più volte.

Milestone 2

Una minaccia alla validità di è legata alla natura delle metriche di accuratezza utilizzate (*Precision, Recall, AUC, Kappa, Accuracy*). Come evidenziato in letteratura, tali metriche forniscono una valutazione della capacità discriminativa del classificatore, ma sono di carattere "estremamente generale". Esse non tengono conto del contesto reale di testing, in cui l'obiettivo primario è massimizzare il numero di bug identificati minimizzando lo sforzo di ispezione (*Effort-Awareness*).

In questo studio, si è scelto di non calcolare la metrica *NPofB20*. Tale scelta è dovuta a un vincolo metodologico stringente: sia la *Ten Times Ten Fold Cross Validation* che l'*Ordered Holdout* generano test set composti da istanze provenienti da release differenti o comunque mescolate. Poiché il calcolo della *NPofB20* richiede che le entità siano ordinate, la natura "mista" dei set impedisce di calcolare tale metrica in modo statisticamente valido. Di conseguenza, si è preferito escludere la metrica piuttosto che produrre valori distorti dall'eterogeneità dei dati.

Per quanto riguarda la metodologia di validazione: la *Ten Times Ten Folds Cross Validation* è intrinsecamente soggetta a una minaccia temporale. Mescolando casualmente le istanze, il modello

rischia di apprendere informazioni da release future per predire difetti in release passate, violando la direzione cronologica dello sviluppo software. Per mitigare questa minaccia, si è affiancata la validazione *Ordered Holdout*, che preserva l'ordine temporale addestrando il modello esclusivamente su dati storici.

Per quanto concerne la generalizzazione dei risultati: lo studio è condotto su progetti open-source del dominio "Apache". Sebbene tale scelta consenta di lavorare su dati di qualità e diversificati, i risultati potrebbero non essere immediatamente estensibili a domini applicativi differenti, dove la distribuzione dei difetti e le pratiche di sviluppo possono variare significativamente.

Milestone 2 – Esperimento personale

Una potenziale minaccia all'esperimento risiede nell'ipotesi formulata riguardo al comportamento dei classificatori. Si è assunto che la rimozione degli identificativi forzasse il modello ad abbandonare la memorizzazione di "scorciatoie" in favore di pattern strutturali. Non si può escludere con certezza matematica che il modello non stia apprendendo correlazioni latenti diverse da quelle ipotizzate. Inoltre esiste il rischio che, con il "Manual Cut", si siano rimosse variabili che contenevano informazioni semantiche rilevanti per la predizione dei difetti.

Milestone 3

Nel contesto dell'analisi What-If, l'inclusione dell'algoritmo SMOTE all'interno della pipeline di classificazione (Random Forest) è stata dettata esclusivamente da un vincolo metodologico di coerenza: l'esperimento richiedeva l'utilizzo della configurazione risultata globalmente più performante durante le fasi di training e testing della precedente Milestone 2. Tuttavia, l'impiego di questa specifica tecnica di bilanciamento introduce due criticità fondamentali che minacciano la validità interna dei risultati ottenuti.

In primo luogo, l'applicazione di SMOTE risulta altamente discutibile da una prospettiva rigorosamente ingegneristica, sollevando fondate opposizioni. A differenza del semplice Oversampling, SMOTE agisce di fatto come un predittore occulto che genera elementi sintetici fittizi estrapolandone le caratteristiche dalle istanze minoritarie reali. L'inserimento di dati generati artificialmente e fuori controllo nel Training Set del classificatore primario inietta un ingiustificabile livello di errore potenziale all'intero processo di apprendimento, allontanando il modello dalla reale distribuzione storica dei difetti.

In secondo luogo, l'impatto di SMOTE ha generato una severa distorsione empirica osservabile nei risultati dello scenario What-If. Il bilanciamento sintetico ha forzato il modello a penalizzare severamente le classi complesse per arginare i falsi negativi, rendendolo di fatto iper-sensibile alla classe *buggy*. Quando il classificatore è stato testato sul Dataset B+ (un sotto-insieme composto esclusivamente dalle classi più grandi e problematiche del sistema, ovvero quelle affette da code smell) il modello ha perso la sua calibrazione originaria, generando un'esplosione incontrollata di falsi positivi (passando da 639 difetti reali a 3706 difetti predetti). Questa anomalia dimostra che un modello addestrato con SMOTE, sebbene apparentemente performante sul dataset globale, rischia di

produrre stime del tutto inaffidabili quando applicato a porzioni di codice isolate e ad alta densità di complessità strutturale.

Milestone 4

L'introduzione dei Large Language Models (LLM) nel processo di refactoring del codice introduce specifiche minacce alla validità di costruito e interna, legate sia alla natura probabilistica degli strumenti sia a limiti operativi durante la fase di prompting.

L'algoritmo di selezione implementato non ha restituito matematicamente la terza e la terzultima classe con il maggior numero di smell in assoluto all'interno dell'intero progetto. Tuttavia, si assume che il divario di complessità tra le due classi effettivamente selezionate resti sufficientemente marcato affinché lo studio comparativo e i risultati ottenuti mantengano la loro validità empirica.

Le metriche e le regole utilizzate per la stima dei code smell sono state concepite su misura per il carico cognitivo di un essere umano, stimando i punti di reale difficoltà di manutenzione per uno sviluppatore in carne ed ossa. Si introduce una minaccia assumendo che queste regole siano altrettanto valide per un LLM: molte architetture potrebbero risultare meno problematiche in funzione della diversa rappresentazione interna del codice che ne fa il modello di intelligenza artificiale.

Non è stato sempre possibile effettuare il refactoring utilizzando un singolo prompt onnicomprensivo a causa delle limitazioni tecniche dello strumento utilizzato (Copilot), il quale non supporta la lettura simultanea di più di 3 file in input. La necessità di fornire i numerosi file di test in modo progressivo e di richiedere all'LLM di reperire autonomamente online il restante contesto del progetto "Syncope" ha frammentato il processo. Questa scelta metodologica incrementa drasticamente il rischio di allucinazioni dovute a contesto mancante o degradato, portando spesso l'IA a dimenticare i vincoli iniziali all'aumentare dei prompt forniti.

La struttura del prompt iniziale ha introdotto istruzioni potenzialmente contraddittorie per il modello. Nello specifico, richiedere di mantenere la segnature pubblica dei metodi assolutamente intatta, domandando contestualmente di risolvere code smell legati all'eccessivo numero di parametri (es. nei costruttori della Classe A), genera un conflitto ineseguibile. In questi casi, il comportamento diventa imprevedibile: è a totale discrezione dell'LLM decidere a quale vincolo dare maggior peso, portando a refactoring parziali o all'elusione completa della richiesta.

Milestone 4 – Esperimento personale

La percentuale di smell rimossi può essere un indicatore di quanto l'LLM abbia compreso la classe ma non è necessariamente un indice di difficoltà. Bisognerebbe approfondire ulteriormente con altri studi.

Il caso che si è verificato è decisamente patologico e non si può ritenere che i risultati siano generalizzabili.

Conclusioni

Il percorso intrapreso in questo progetto di Ingegneria del Software, focalizzato sul caso studio di *Apache Syncope*, si è rivelato un'esperienza formativa complessa e multidisciplinare. Si è partiti dalla gestione di dati grezzi provenienti da repository Git e Jira, superando le insidie metodologiche del labeling e della normalizzazione dei dati, fino ad arrivare all'applicazione avanzata di modelli di Machine Learning e all'interazione con gli LLM per il refactoring automatico.

Ciò che ha reso questo percorso particolarmente stimolante è stata la possibilità di non limitarsi all'esecuzione delle milestone previste, ma di affiancarvi una componente di ricerca personale. Sono stati effettuati esperimenti che inizialmente non erano contemplati, ma che hanno dato valore aggiunto alla comprensione profonda dei modelli.

I risultati ottenuti confermano che la defect prediction è uno strumento di supporto potente, ma non privo di zone d'ombra. Si è dimostrato che:

1. ***Lo "Smell" è solo la punta dell'iceberg:*** l'analisi *What-If* ha dimostrato che rimuovere code smell senza modificare la struttura profonda del sistema (es. LOC, Churn) è un'operazione estetica che potrebbe non intaccare la reale stabilità del codice.
2. ***L'IA va guidata, non seguita ciecamente:*** l'esperimento sulle classi rifattorizzate ha evidenziato il rischio concreto di "allucinazioni semantiche". L'LLM può generare codice che compila e passa i test, ma che altera silenziosamente il contratto delle eccezioni, rendendo il sistema meno robusto di prima.

In conclusione, il messaggio chiave è che non esistono *silver bullets*. Né il Machine Learning né l'IA generativa possono sostituire il giudizio critico dell'ingegnere. L'automazione è un acceleratore, ma richiede una validazione rigorosa. I futuri sviluppi di questo lavoro potrebbero puntare su una validazione ***semantica*** automatica del codice generato dagli LLM, per evitare che un refactoring automatico introduca silenziosamente bug o bug regressivi. Lo sviluppo software non è mera scrittura di codice, ma gestione consapevole della qualità e dei dati. Si ritiene che, spesso, le risposte più interessanti arrivano proprio quando si decide di provare a rompere gli schemi prefissati.

Riferimenti

Link GitHub

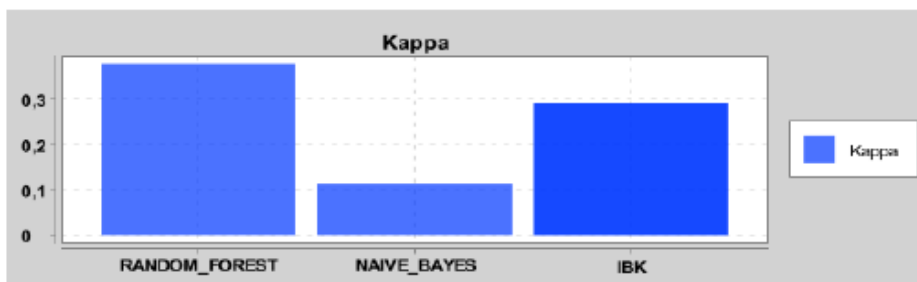
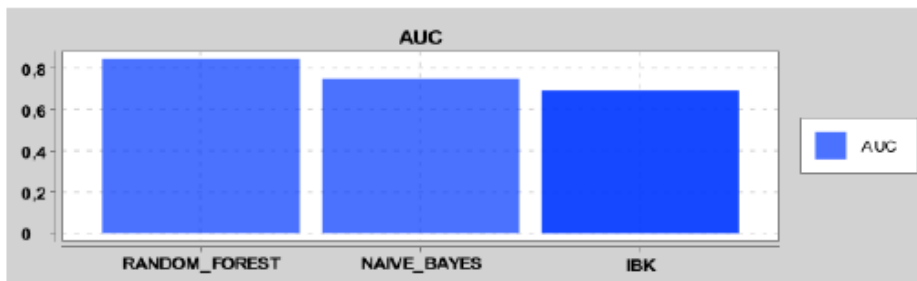
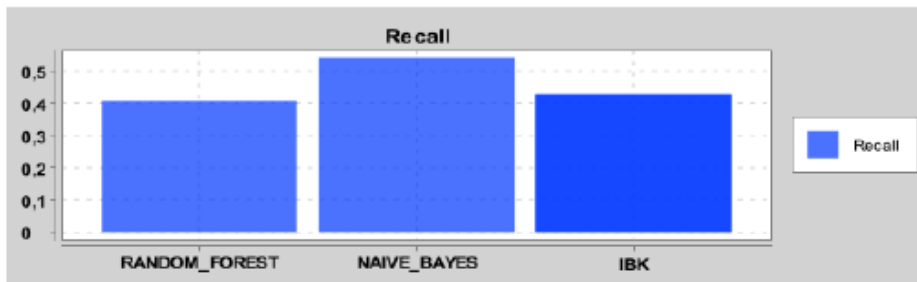
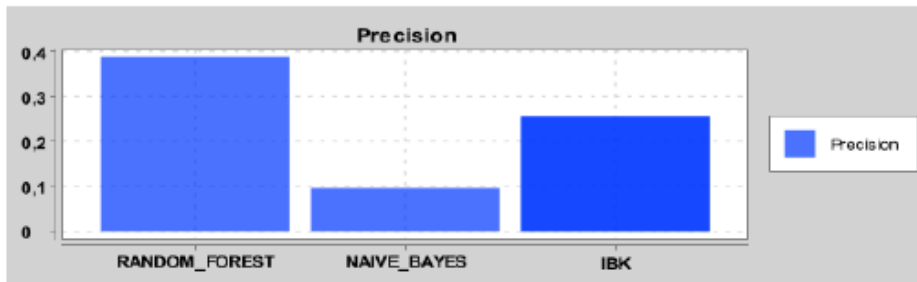
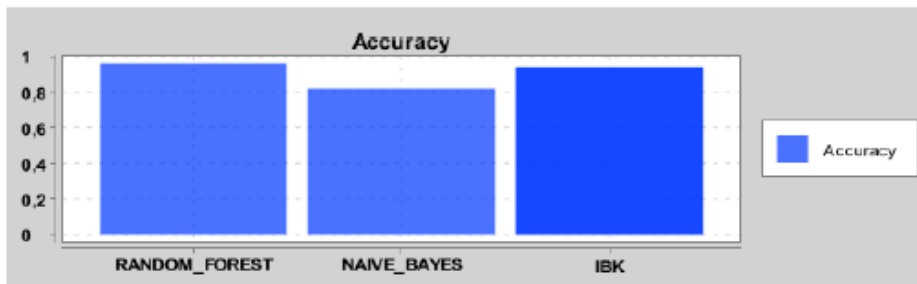
<https://github.com/gianpaolo-pestilli/dataset.git>

Link SonarCloud

https://sonarcloud.io/project/overview?id=gianpaolo-pestilli_dataset

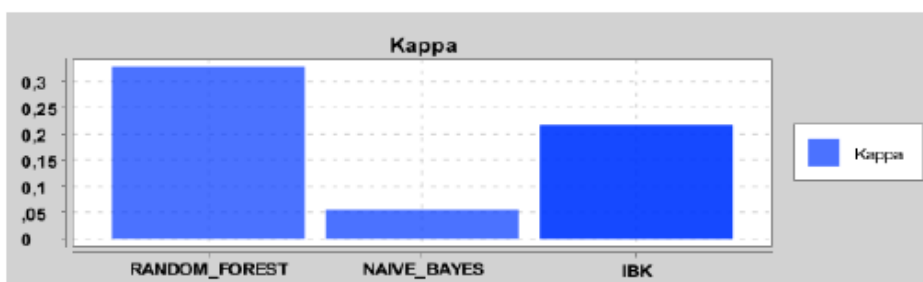
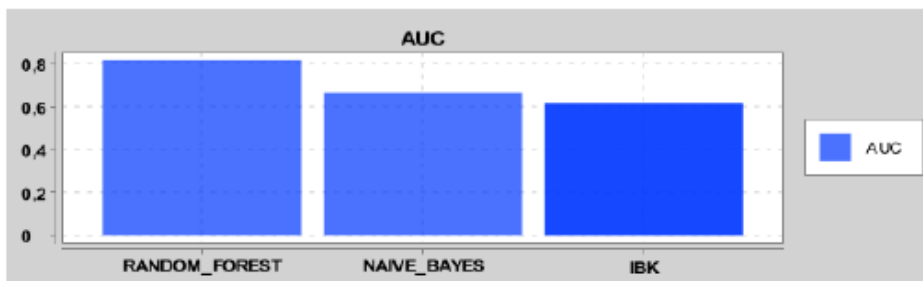
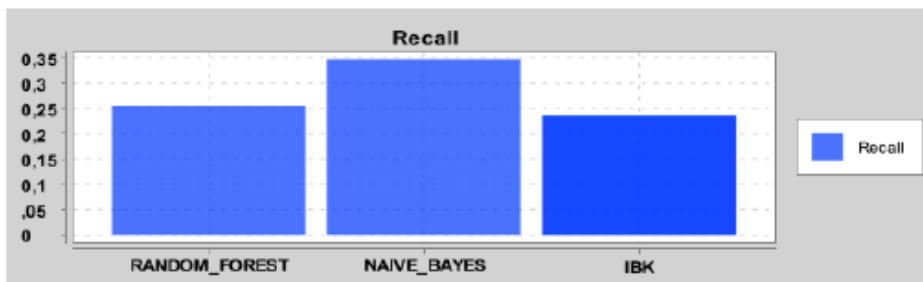
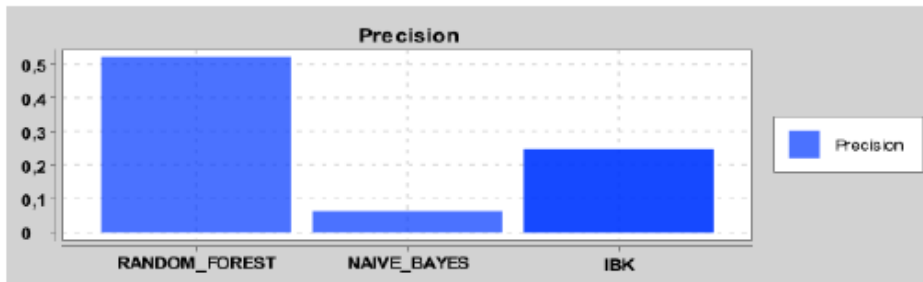
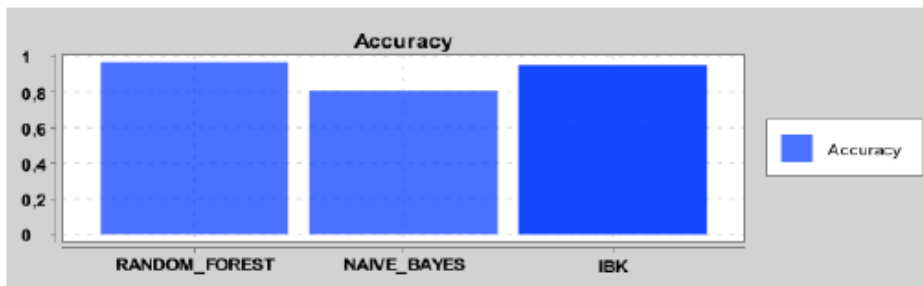
Appendice

CUT: NO; FEATURE SELECTION: FILTER; BALANCING: NO; VALIDATION: OH:



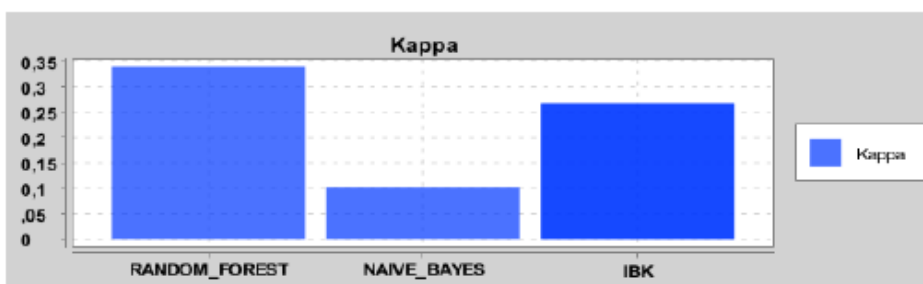
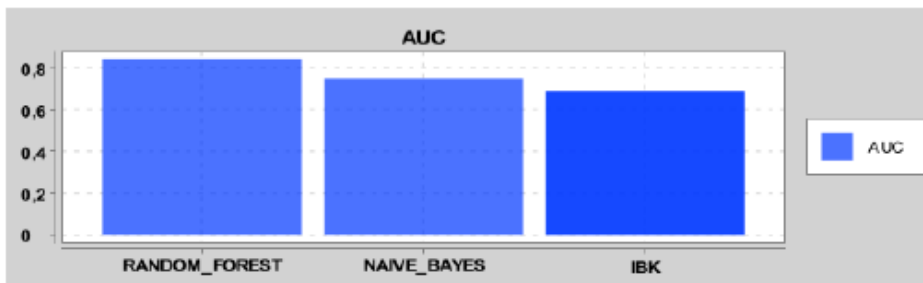
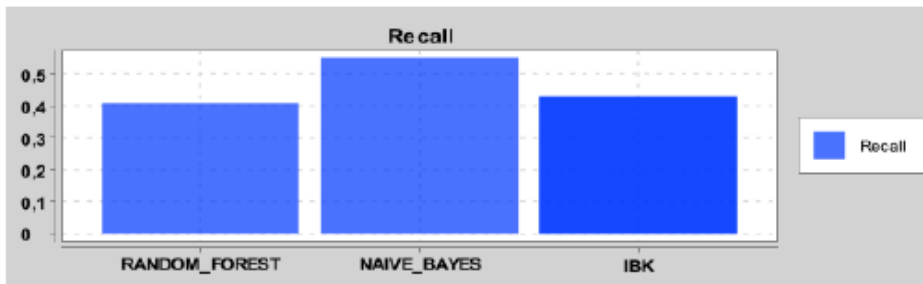
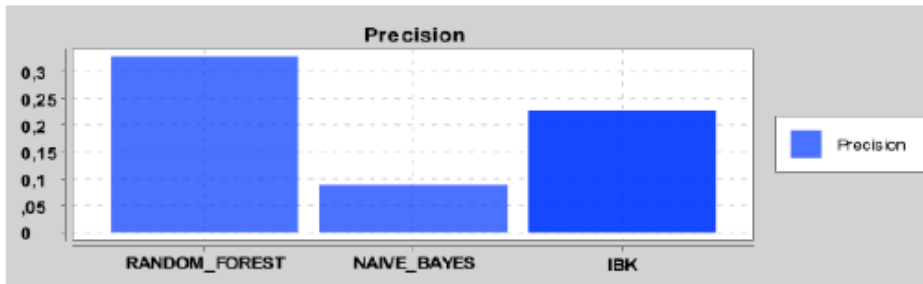
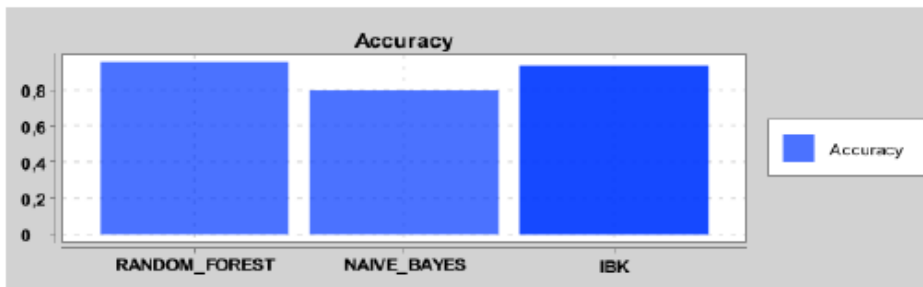
Classifier	TP	FP	TN	FN
RANDOM_FOREST	40	63	2830	58
NAIVE_BAYES	53	497	2396	45
IBK	42	122	2771	56

CUT: YES; FEATURE SELECTION: NO; BALANCING: NO; VALIDATION: OH:



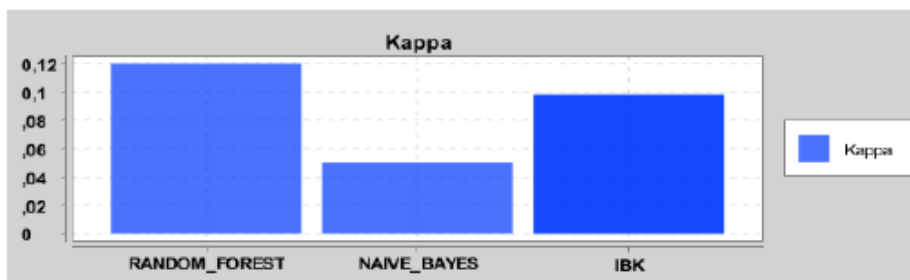
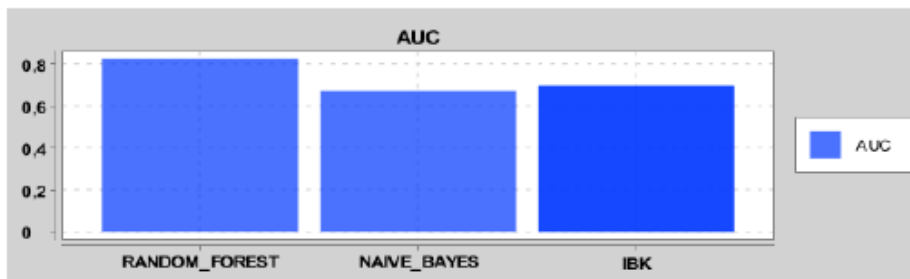
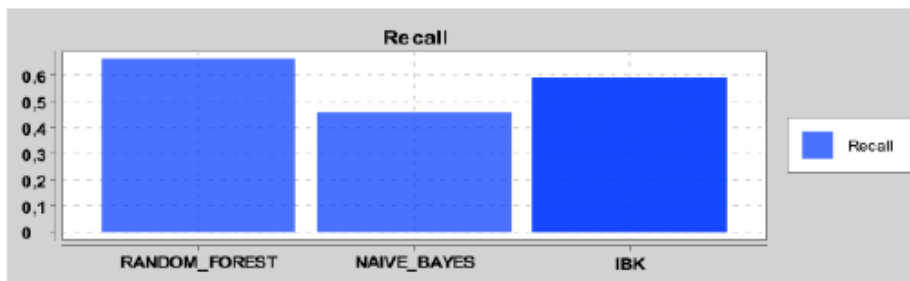
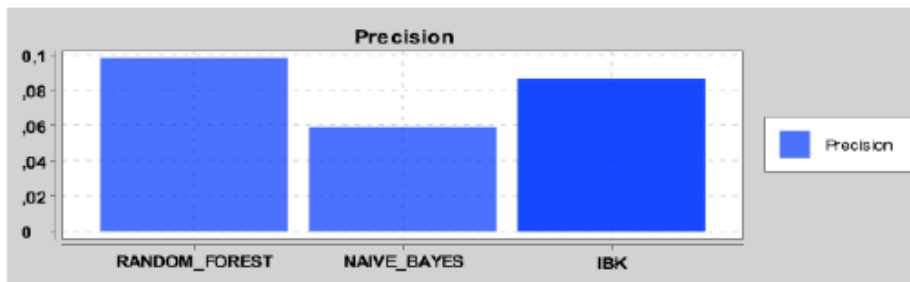
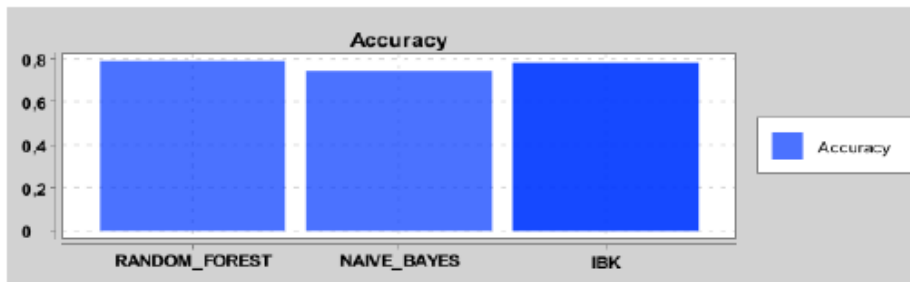
Classifier	TP	FP	TN	FN
RANDOM_FOREST	25	23	2870	73
NAIVE_BAYES	34	501	2392	64
IBK	23	70	2823	75

CUT: NO; FEATURE SELECTION: FILTER; BALANCING: SMOTE; VALIDATION: OH:



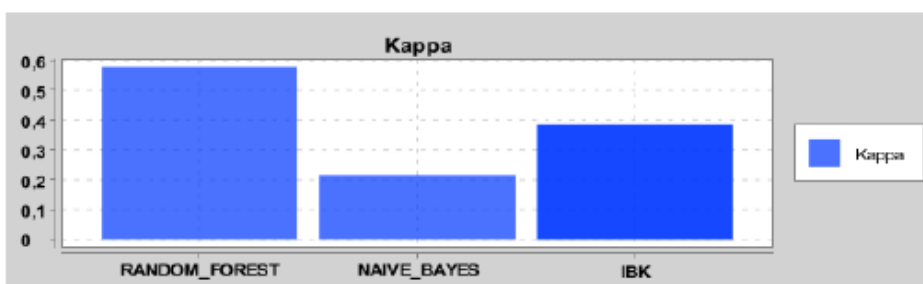
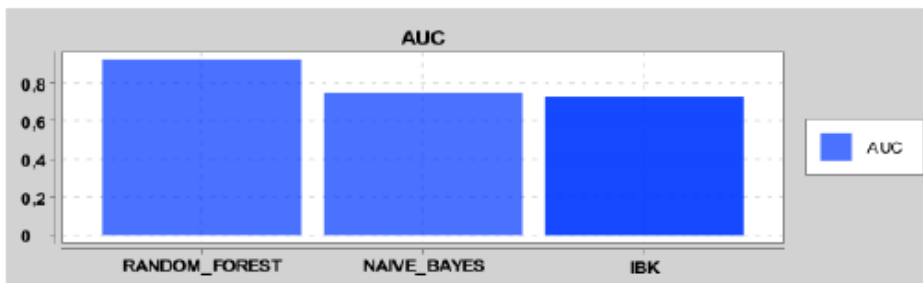
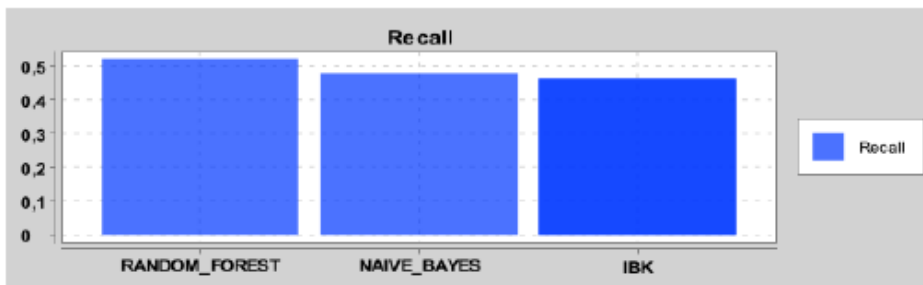
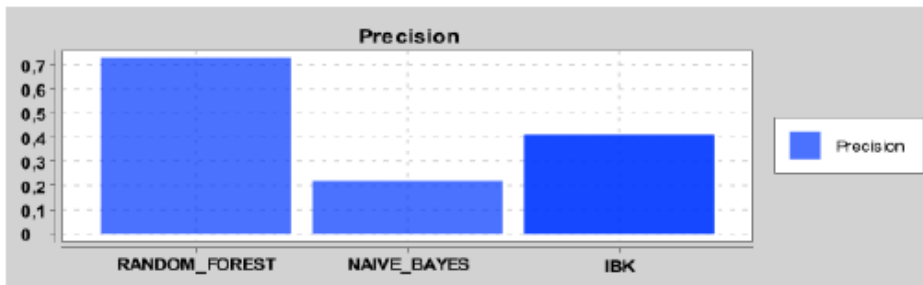
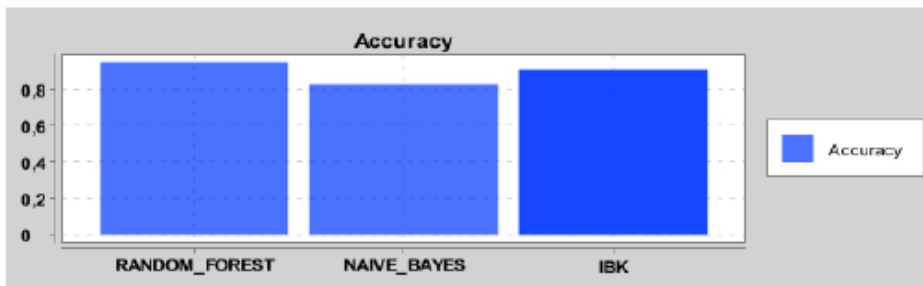
Classifier	TP	FP	TN	FN
RANDOM_FOREST	40	82	2811	58
NAIVE_BAYES	54	553	2340	44
IBK	42	142	2751	56

CUT: YES; FEATURE SELECTION: NO; BALANCING: UNDERSAMPLING; VALIDATION: OH:



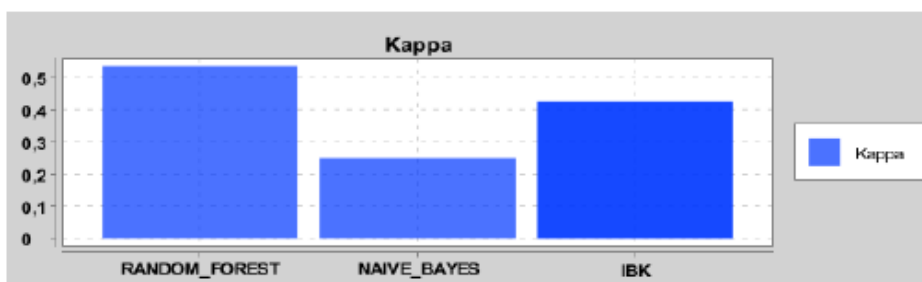
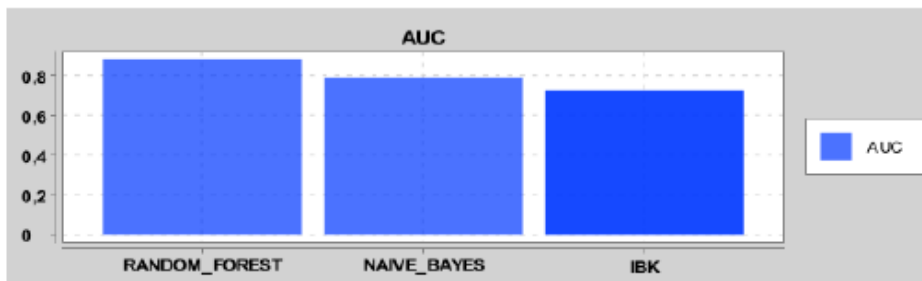
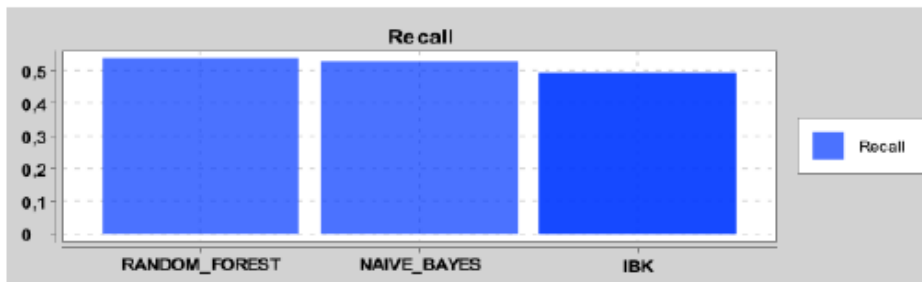
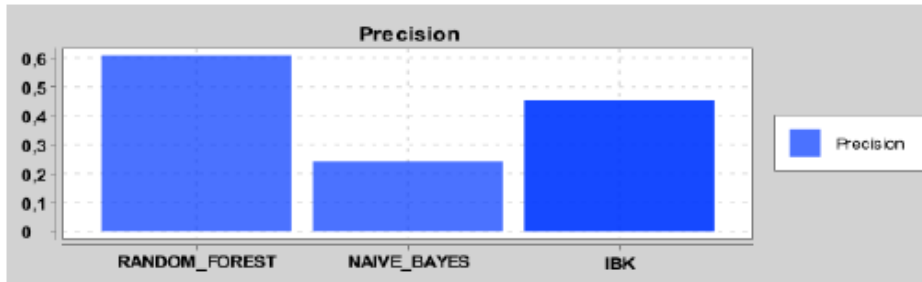
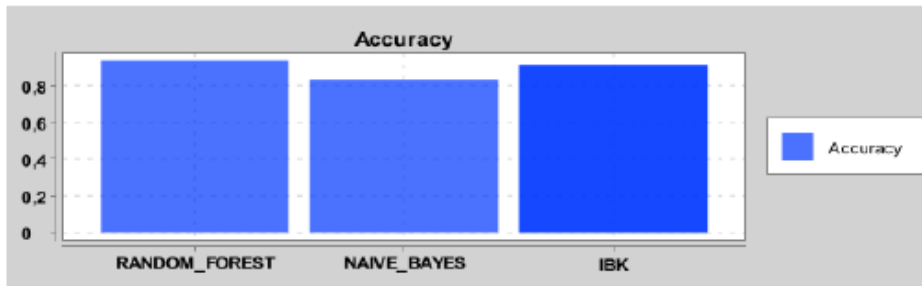
Classifier	TP	FP	TN	FN
RANDOM_FOREST	65	599	2294	33
NAIVE_BAYES	45	713	2180	53
IBK	58	619	2274	40

CUT: YES; FEATURE SELECTION: FILTER; BALANCING: SMOTE; VALIDATION: TTTFCV:



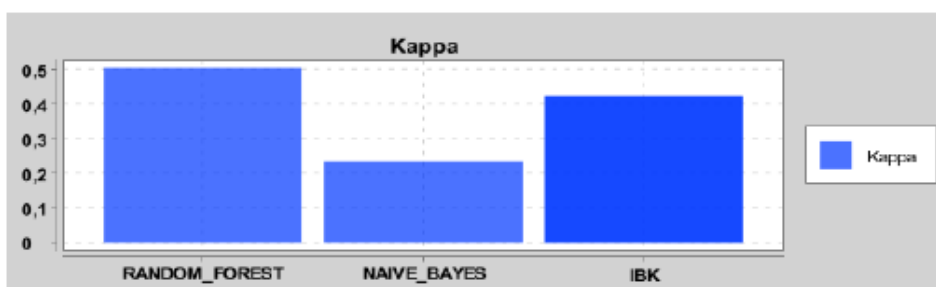
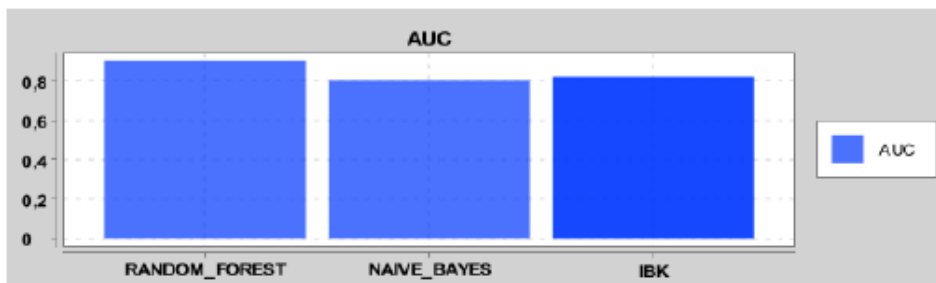
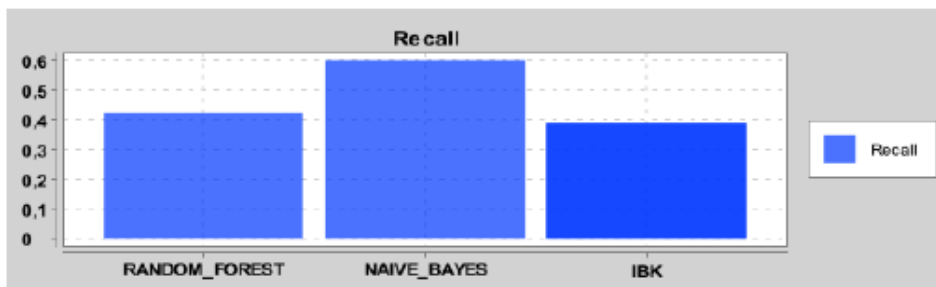
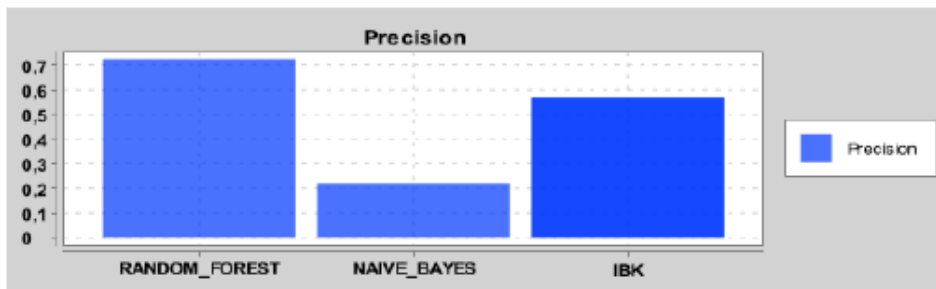
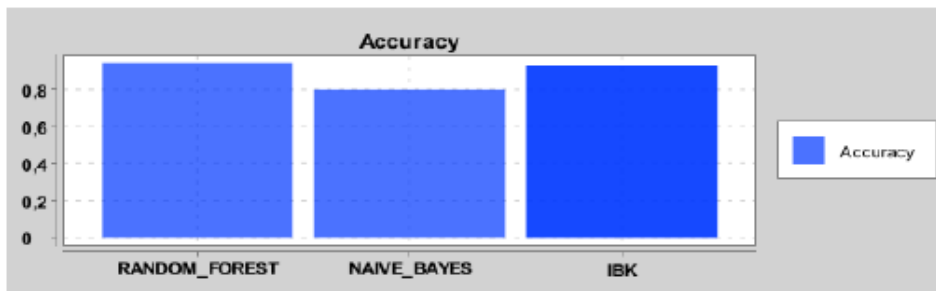
Classifier	TP	FP	TN	FN
RANDOM_FOREST	6125	2285	135475	5665
NAIVE_BAYES	5632	19975	117785	6158
IBK	5461	7830	129930	6329

CUT: NO; FEATURE SELECTION: FILTER; BALANCING: SMOTE; VALIDATION: TTFCV:



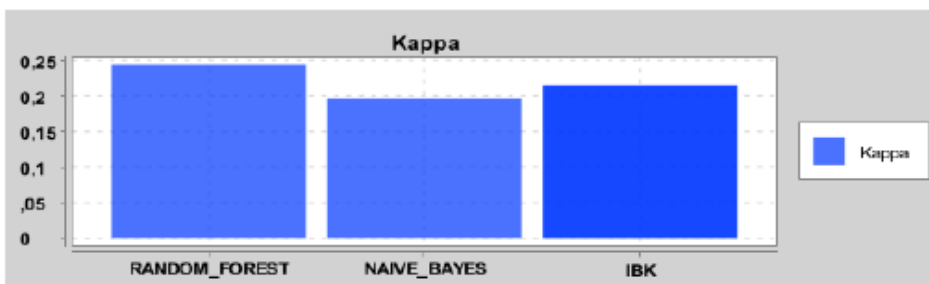
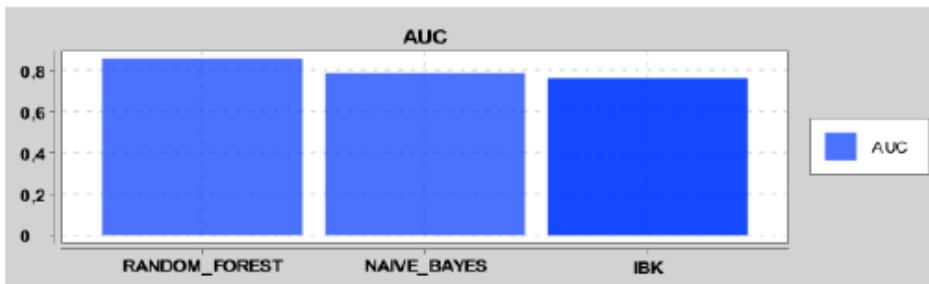
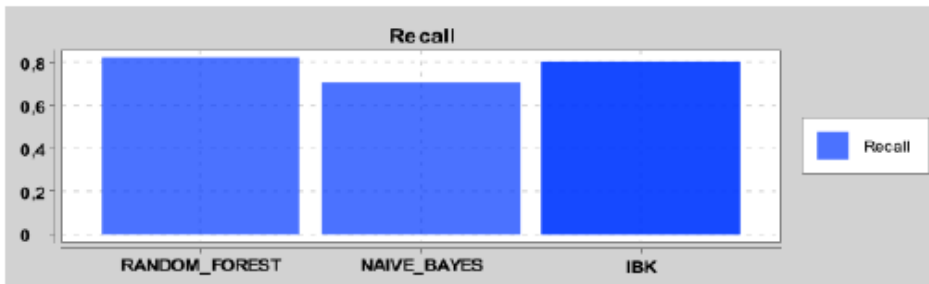
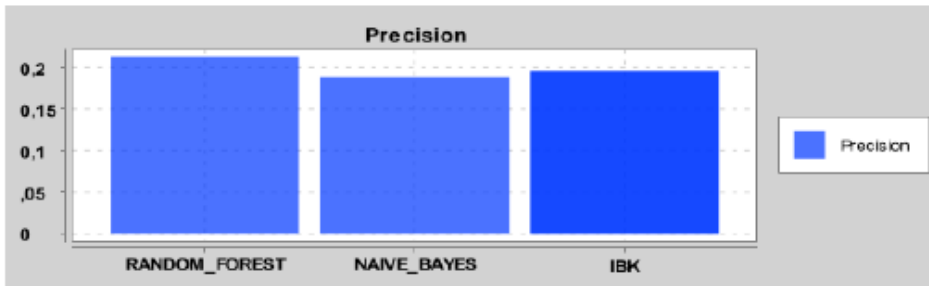
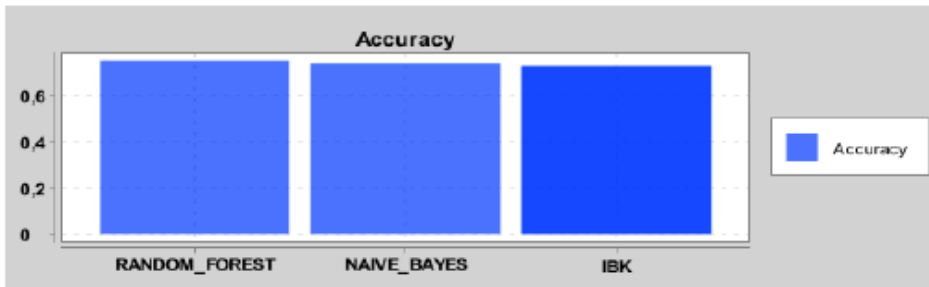
Classifier	TP	FP	TN	FN
RANDOM_FOREST	6335	4108	133652	5455
NAIVE_BAYES	6222	19646	118114	5568
IBK	5803	7051	130709	5987

CUT: NO; FEATURE SELECTION: NO; BALANCING: NO; VALIDATION: TTTCV:



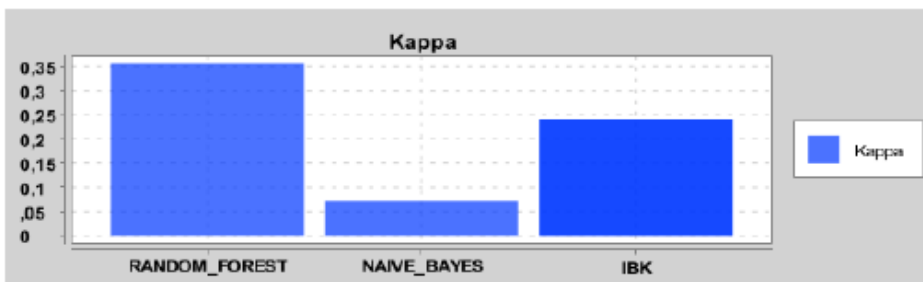
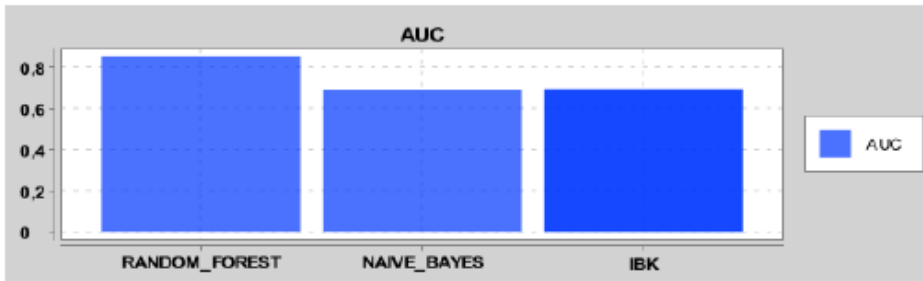
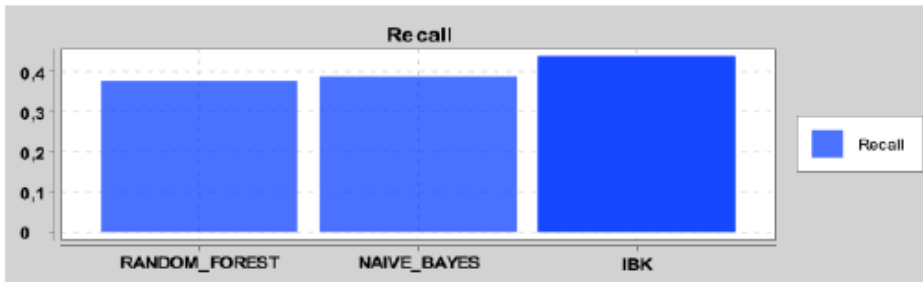
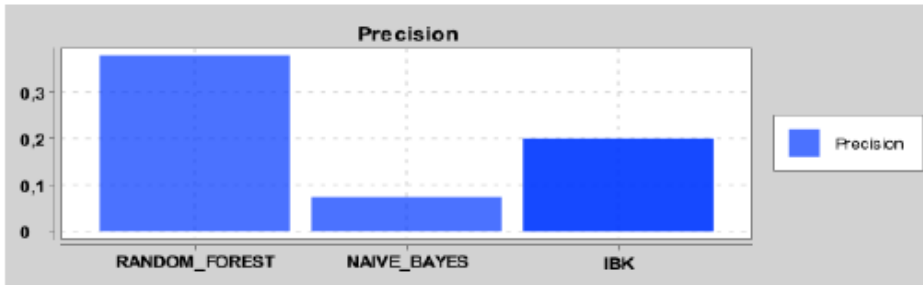
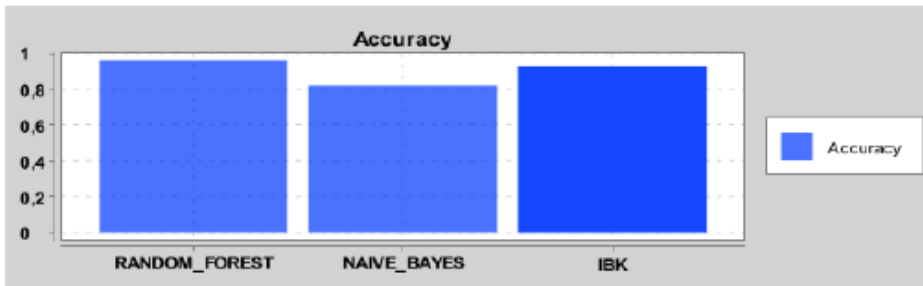
Classifier	TP	FP	TN	FN
RANDOM_FOREST	4976	1902	135858	6814
NAIVE_BAYES	7068	25115	112645	4722
IBK	4562	3447	134313	7228

CUT: NO; FEATURE SELECTION: FILTER; BALANCING: UNDERSAMPLING; VALIDATION: TTFCV:



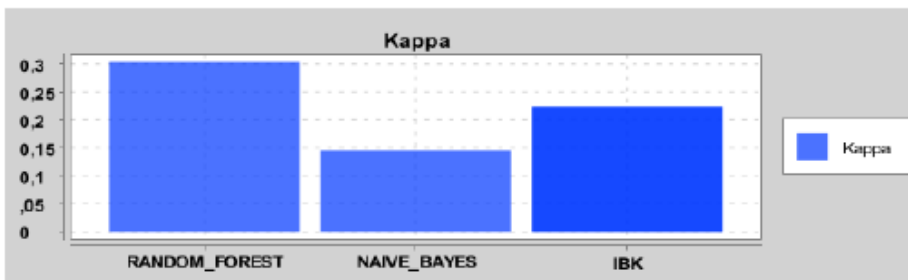
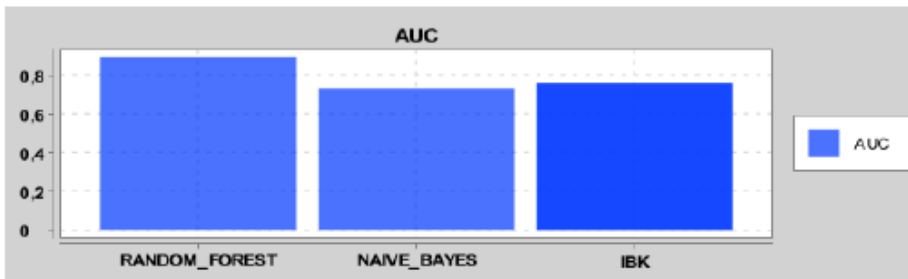
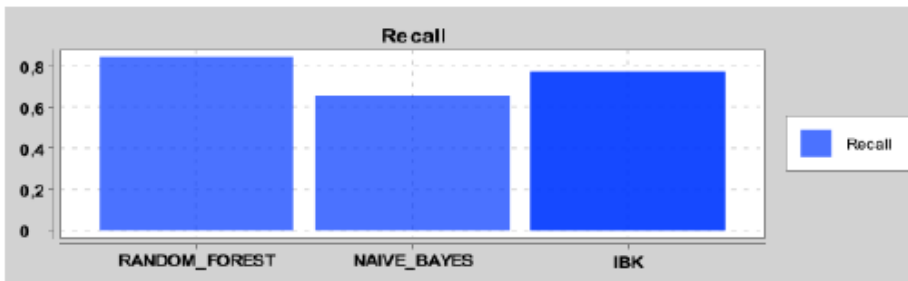
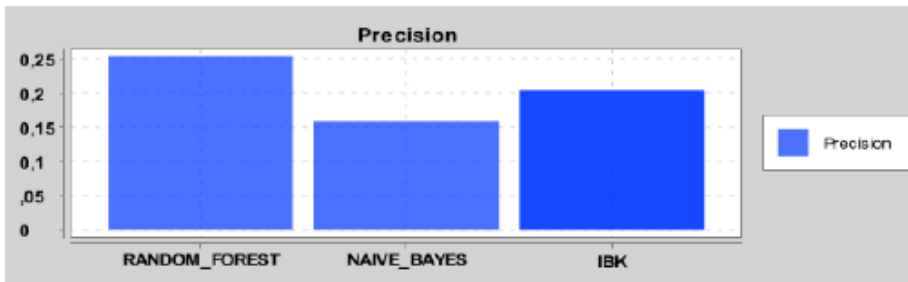
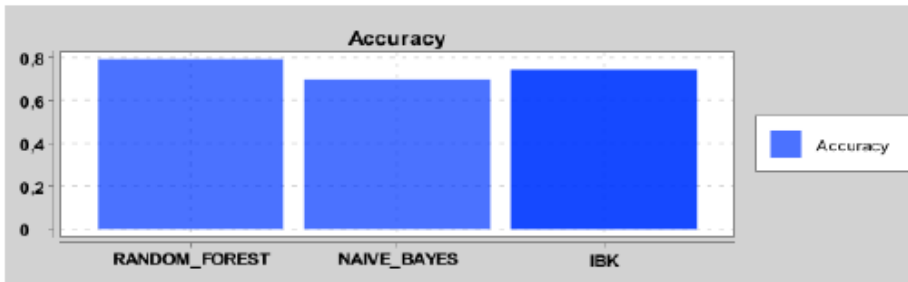
Classifier	TP	FP	TN	FN
RANDOM_FOREST	9694	35783	101977	2096
NAIVE_BAYES	8334	36045	101715	3456
IBK	9443	38850	98910	2347

CUT: NO; FEATURE SELECTION: NO; BALANCING: SMOTE; VALIDATION: OH:



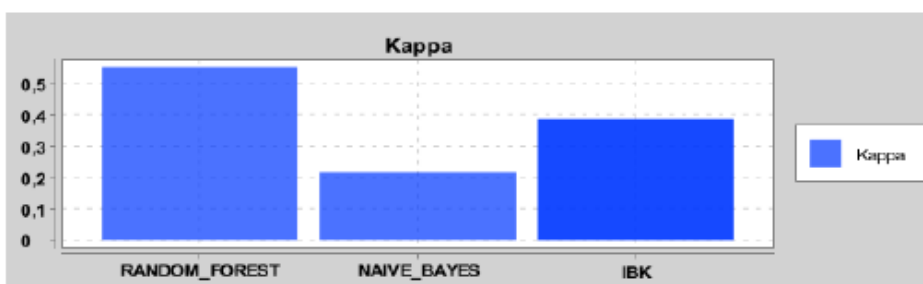
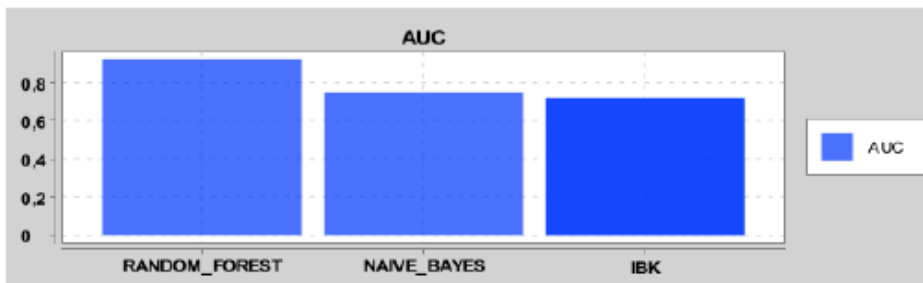
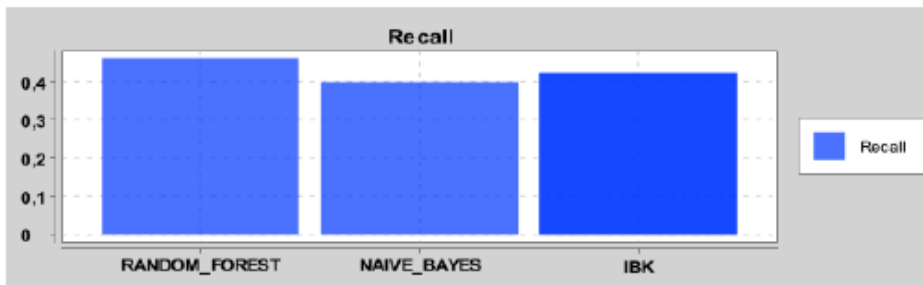
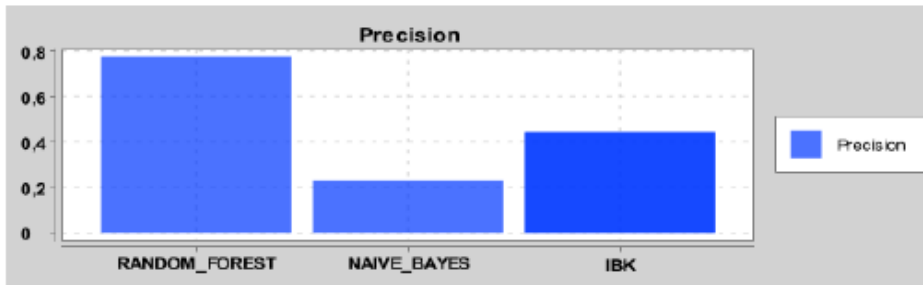
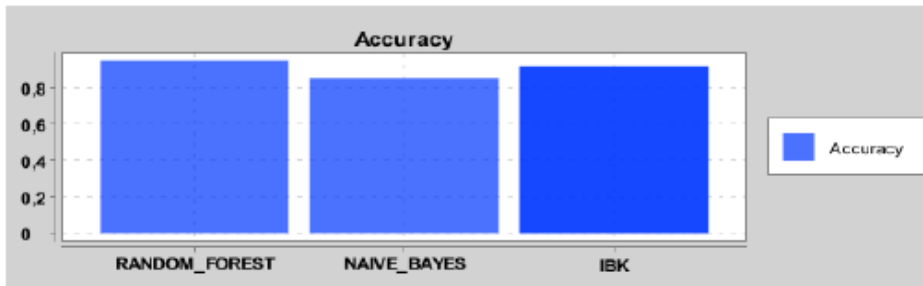
Classifier	TP	FP	TN	FN
RANDOM_FOREST	37	61	2832	61
NAIVE_BAYES	38	480	2413	60
IBK	43	172	2721	55

CUT: YES; FEATURE SELECTION: NO; BALANCING: UNDERSAMPLING; VALIDATION: TTFCV:



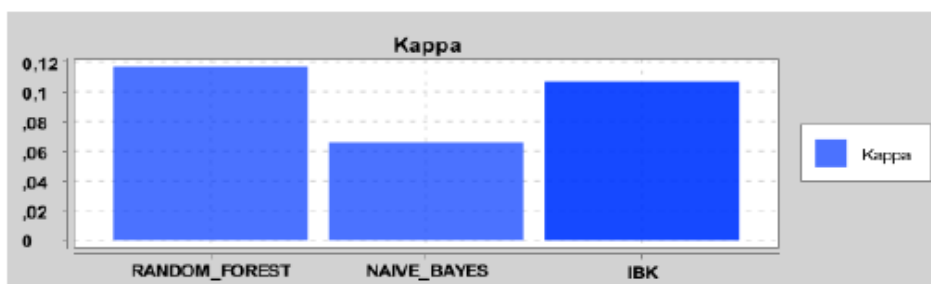
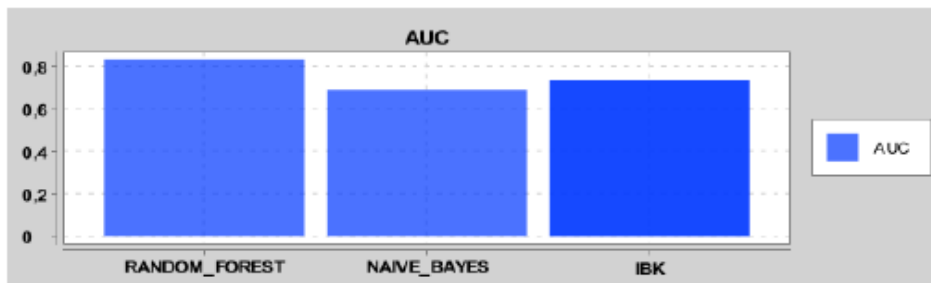
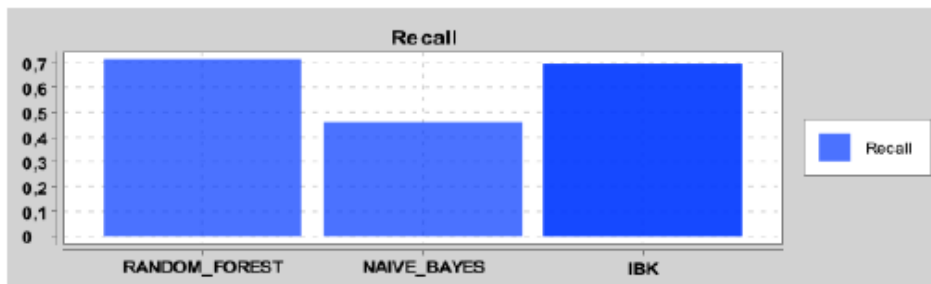
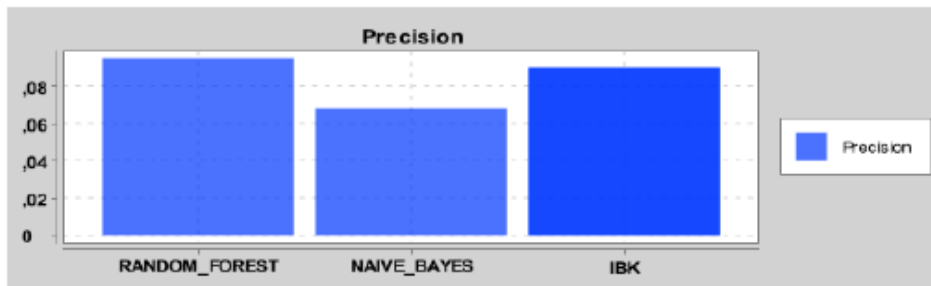
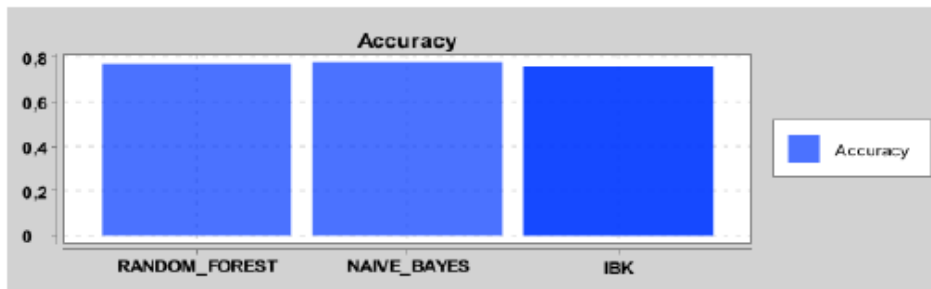
Classifier	TP	FP	TN	FN
RANDOM_FOREST	9951	29442	108318	1839
NAIVE_BAYES	7709	41035	96725	4081
IBK	9092	35798	101962	2698

CUT: YES; FEATURE SELECTION: FILTER; BALANCING: NO; VALIDATION: TTFCV:



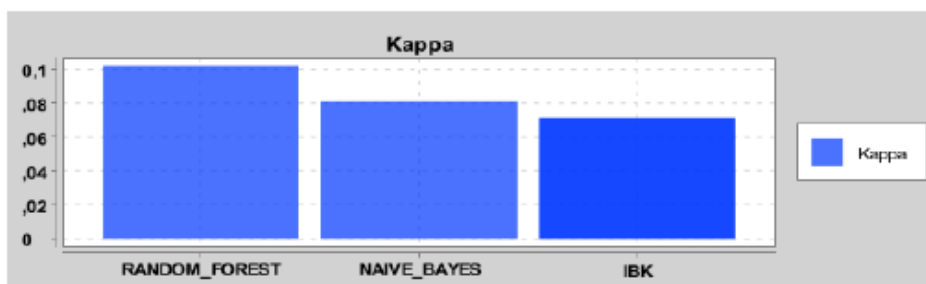
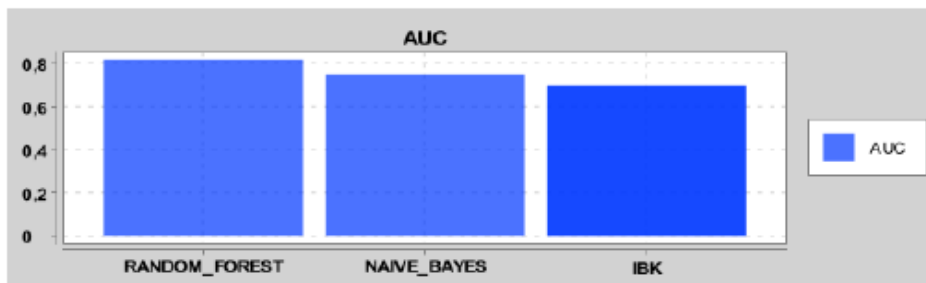
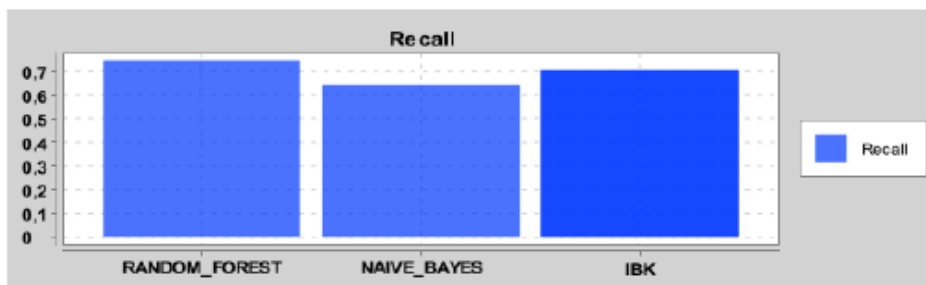
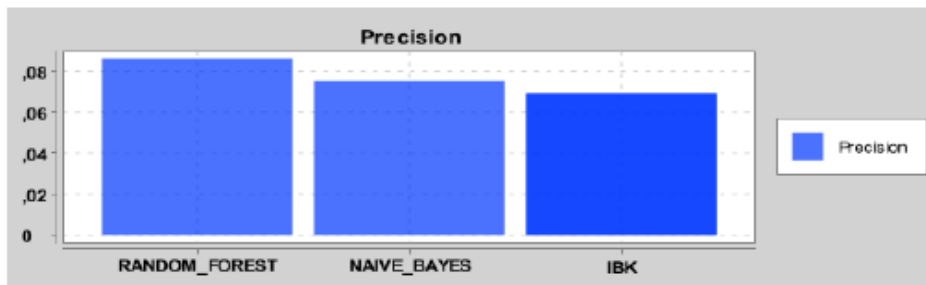
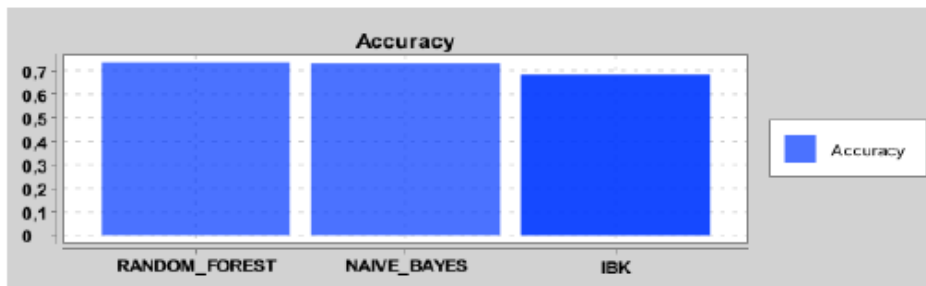
Classifier	TP	FP	TN	FN
RANDOM_FOREST	5437	1575	136185	6353
NAIVE_BAYES	4704	15591	122169	7086
IBK	4980	6253	131507	6810

CUT: NO; FEATURE SELECTION: NO; BALANCING: UNDERSAMPLING; VALIDATION: OH:



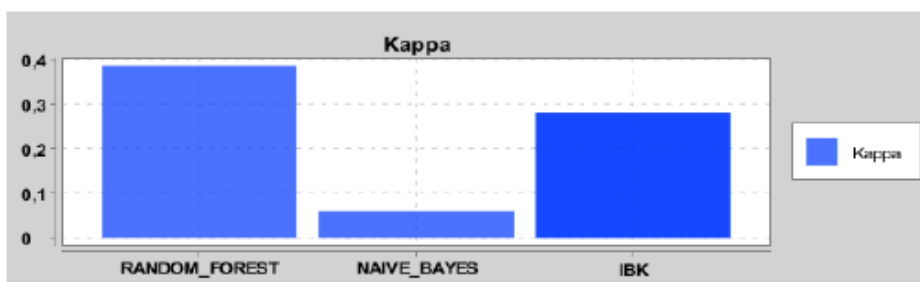
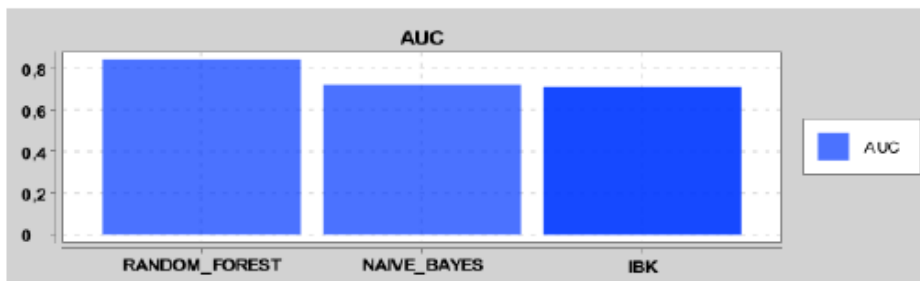
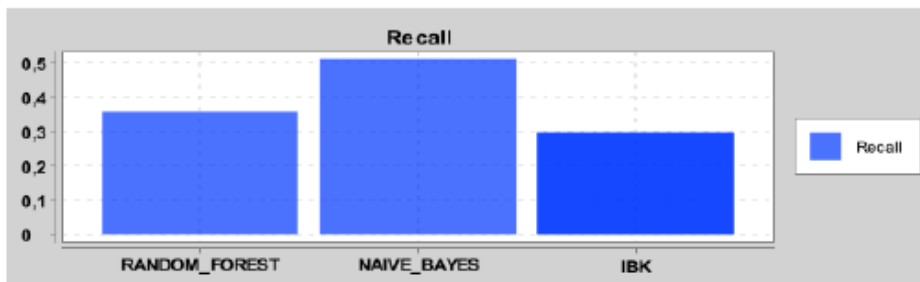
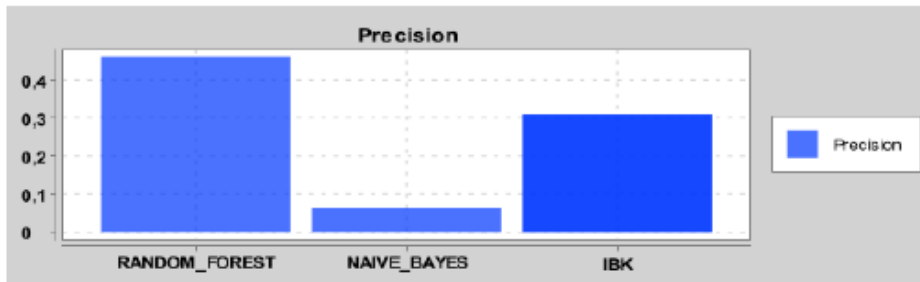
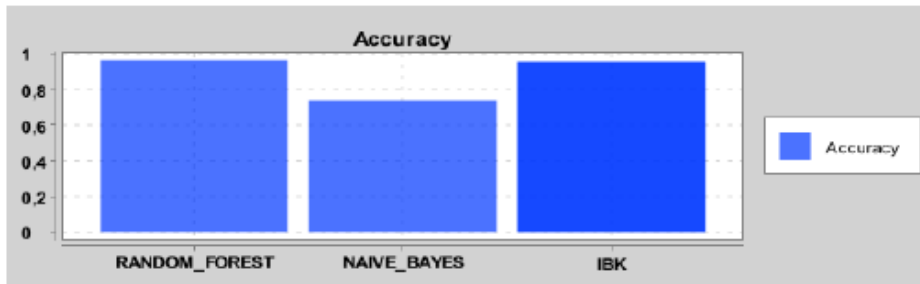
Classifier	TP	FP	TN	FN
RANDOM_FOREST	70	663	2230	28
NAIVE_BAYES	45	613	2280	53
IBK	68	689	2204	30

CUT: NO; FEATURE SELECTION: FILTER; BALANCING: UNDERSAMPLING; VALIDATION: OH:



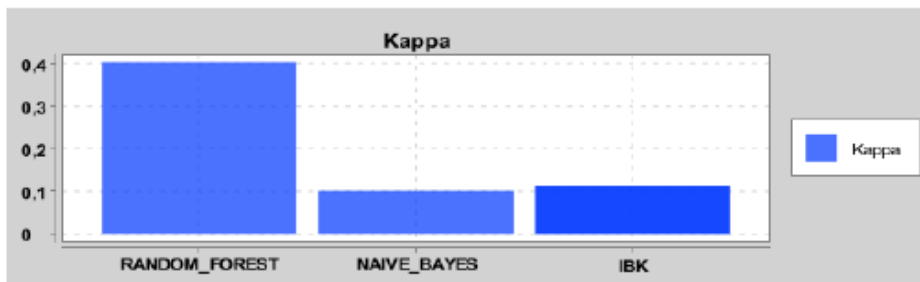
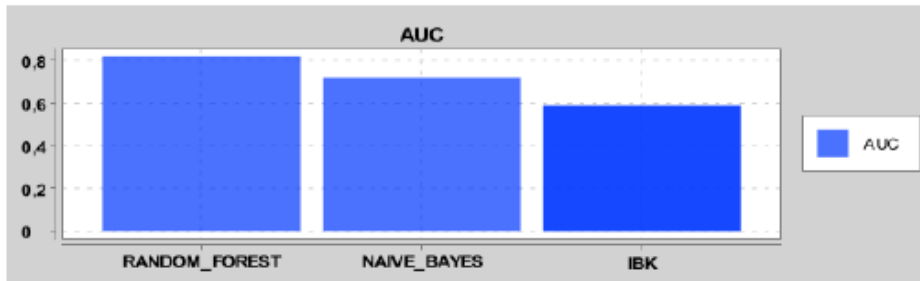
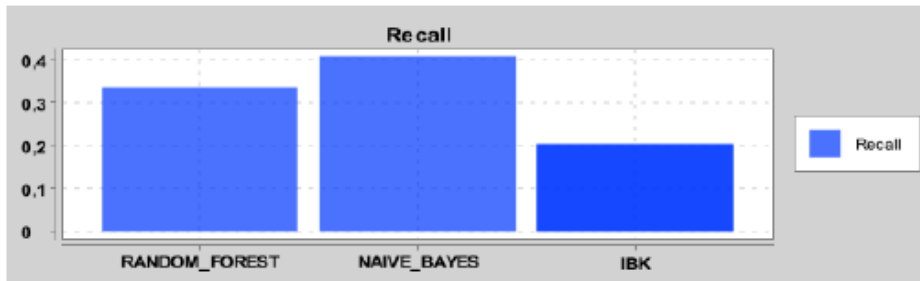
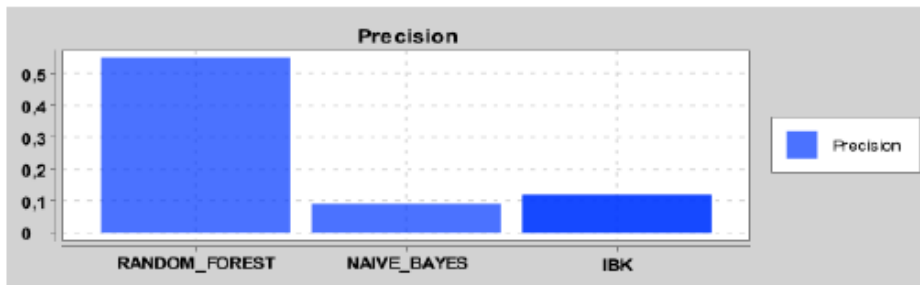
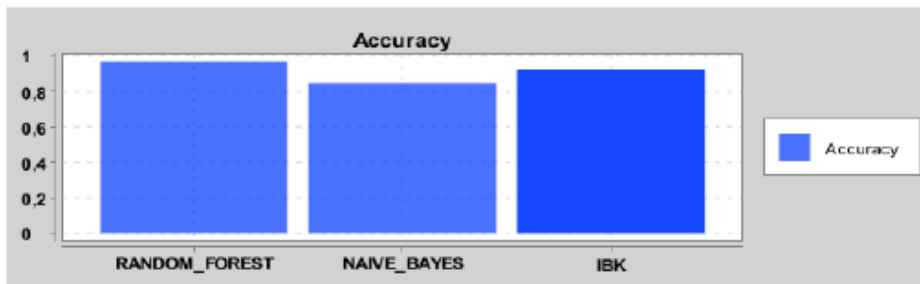
Classifier	TP	FP	TN	FN
RANDOM_FOREST	73	772	2121	25
NAIVE_BAYES	63	775	2118	35
IBK	69	928	1965	29

CUT: NO; FEATURE SELECTION: NO; BALANCING: NO; VALIDATION: OH:



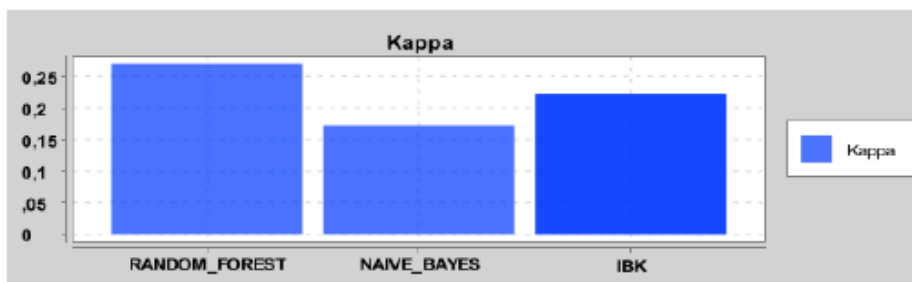
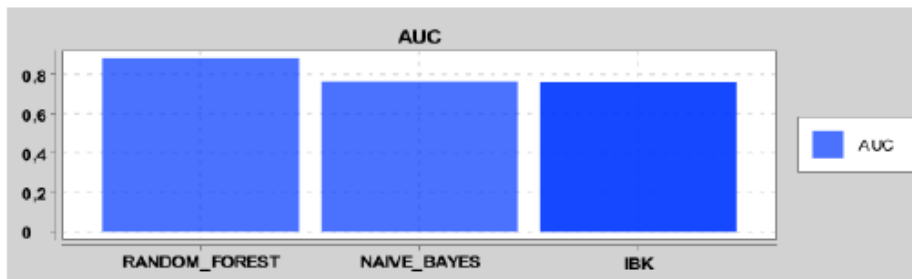
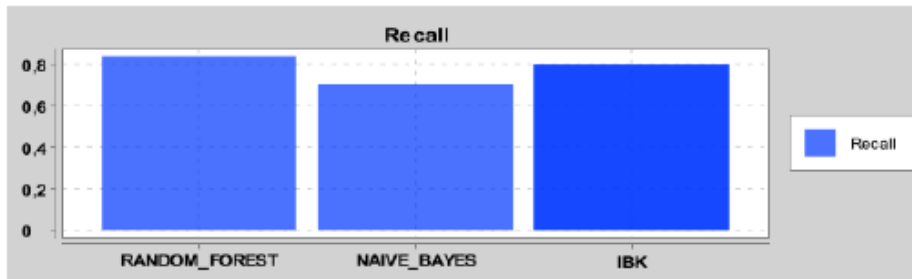
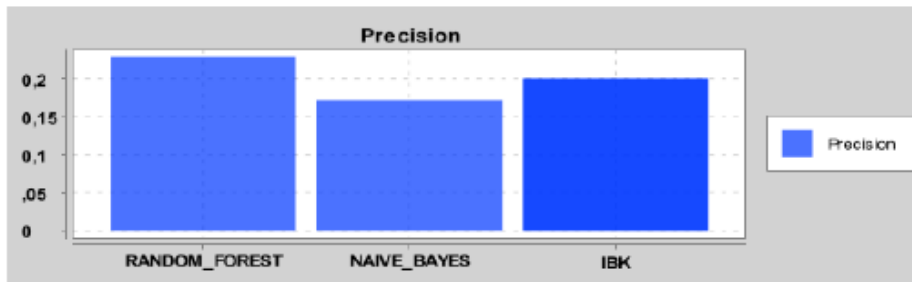
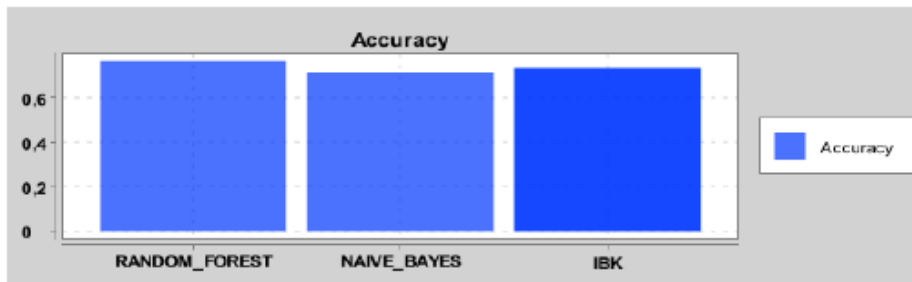
Classifier	TP	FP	TN	FN
RANDOM_FOREST	35	41	2852	63
NAIVE_BAYES	50	734	2159	48
IBK	29	65	2828	69

CUT: YES; FEATURE SELECTION: FILTER; BALANCING: SMOTE; VALIDATION: OH:



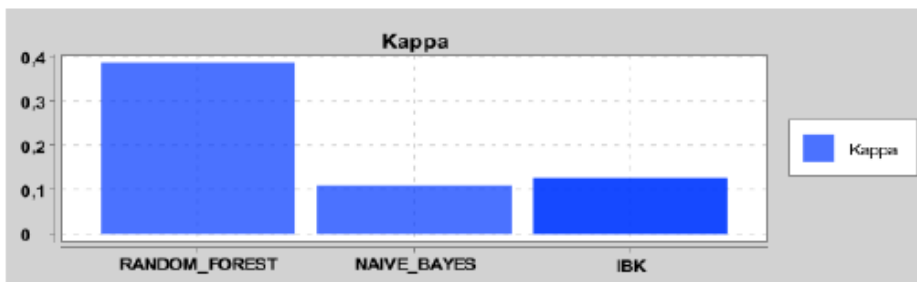
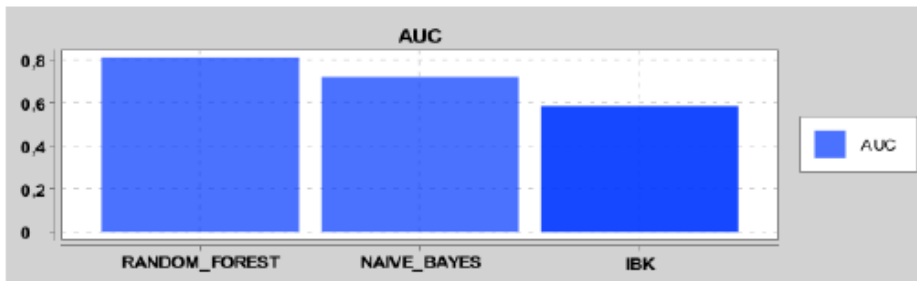
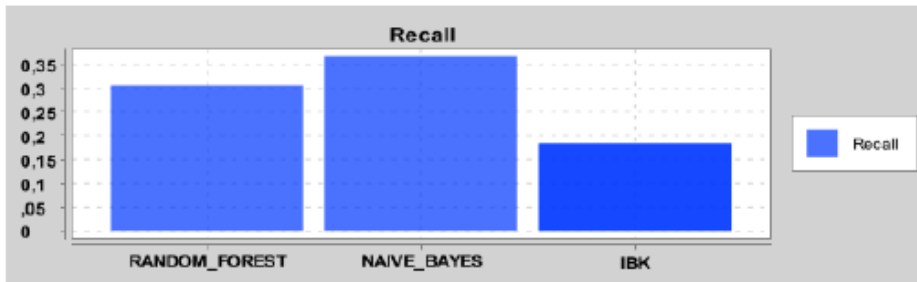
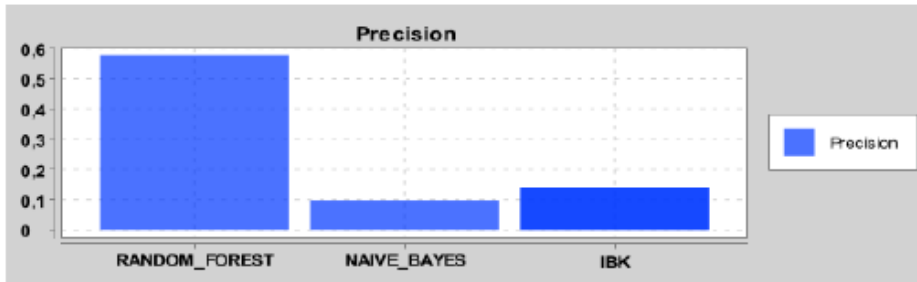
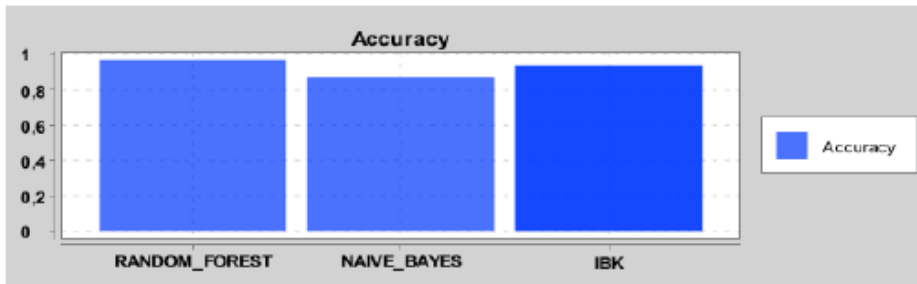
Classifier	TP	FP	TN	FN
RANDOM_FOREST	33	27	2866	65
NAIVE_BAYES	40	399	2494	58
IBK	20	149	2744	78

CUT: NO; FEATURE SELECTION: NO; BALANCING: UNDERSAMPLING; VALIDATION: TTTCV:



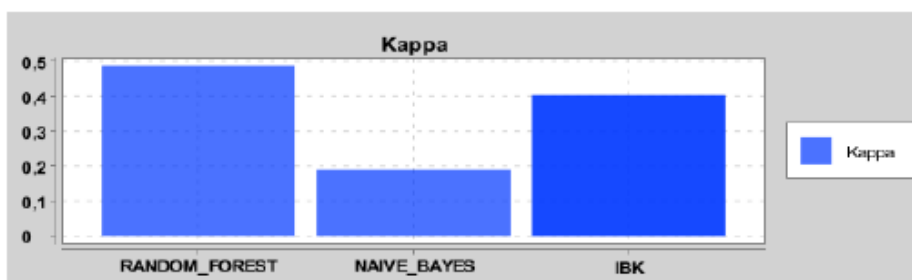
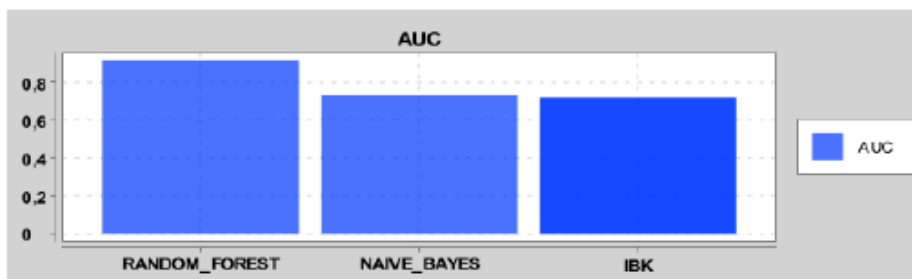
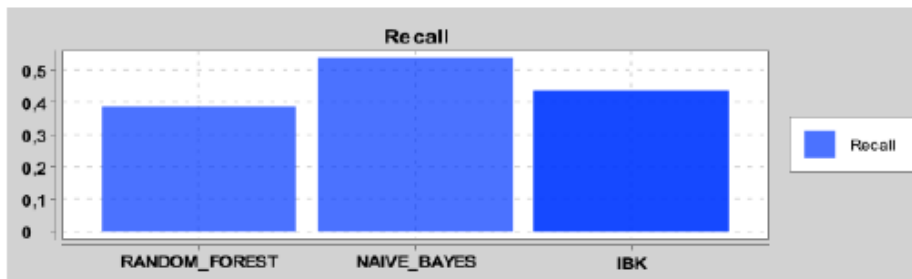
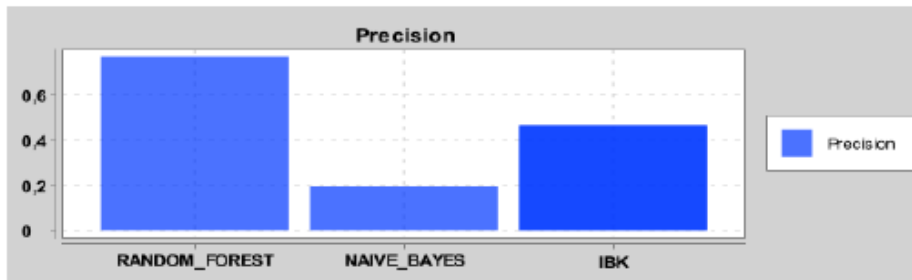
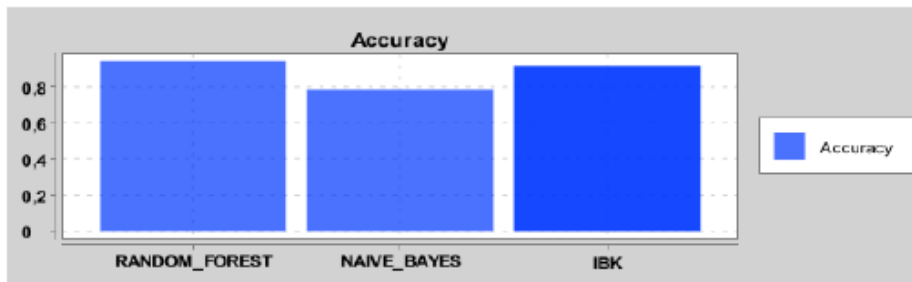
Classifier	TP	FP	TN	FN
RANDOM_FOREST	9870	33163	104597	1920
NAIVE_BAYES	8272	39724	98036	3518
IBK	9437	37729	100031	2353

CUT: YES; FEATURE SELECTION: FILTER; BALANCING: NO; VALIDATION: OH:



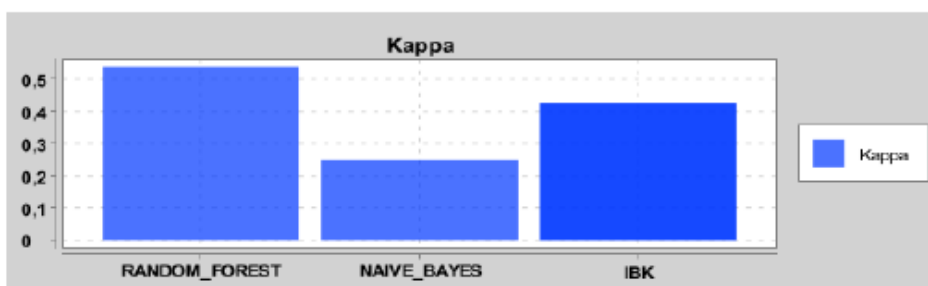
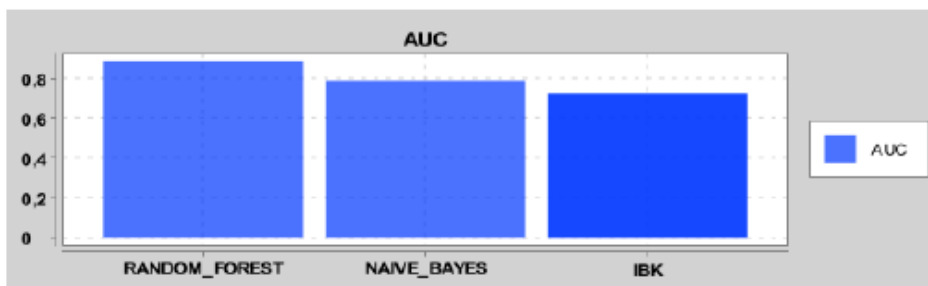
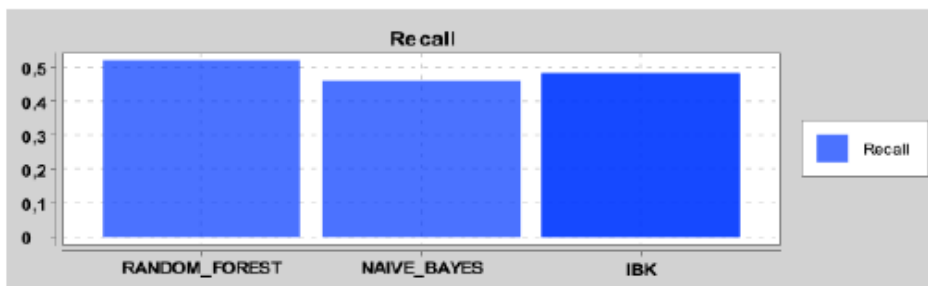
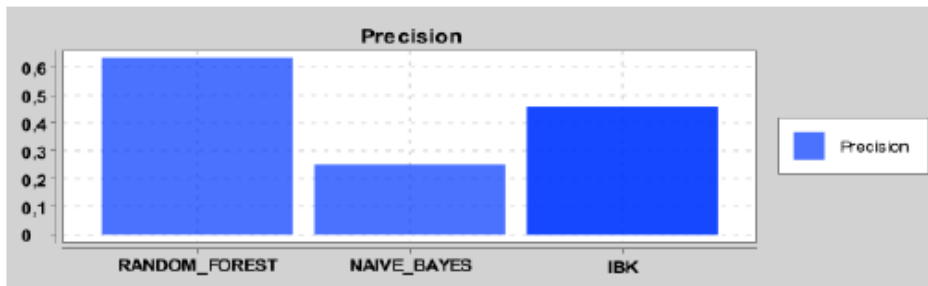
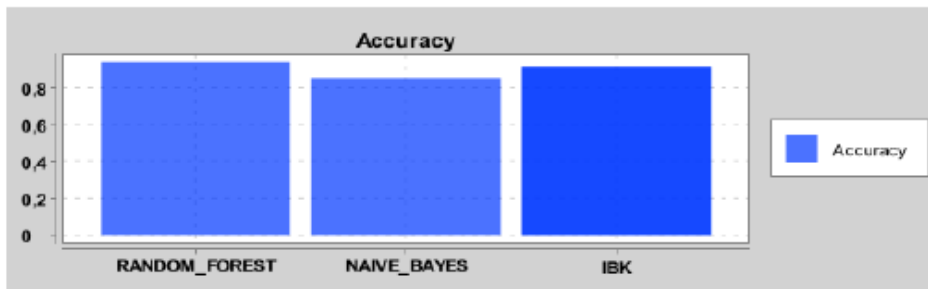
Classifier	TP	FP	TN	FN
RANDOM_FOREST	30	22	2871	68
NAIVE_BAYES	36	331	2562	62
IBK	18	111	2782	80

CUT: YES; FEATURE SELECTION: NO; BALANCING: NO; VALIDATION: TTFCV:



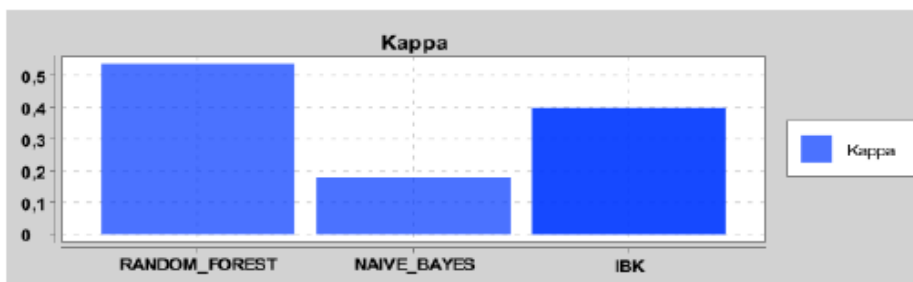
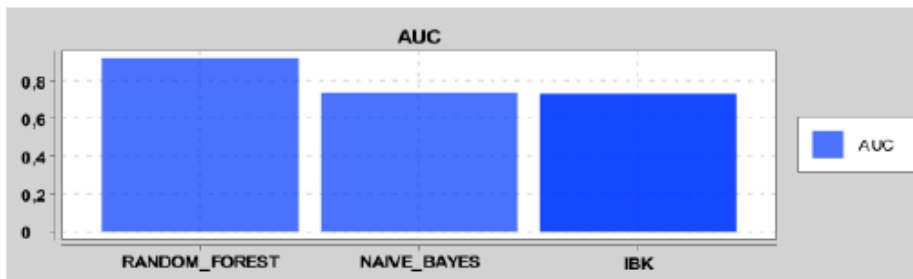
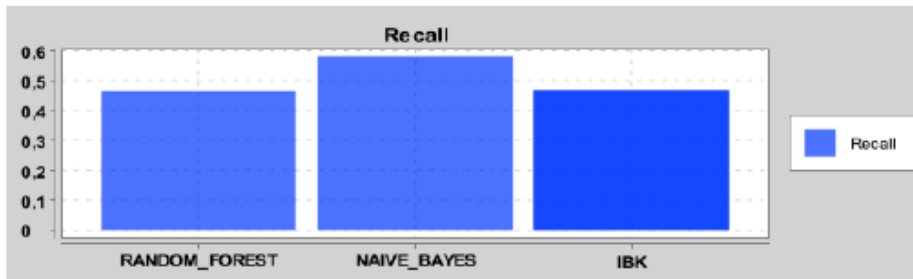
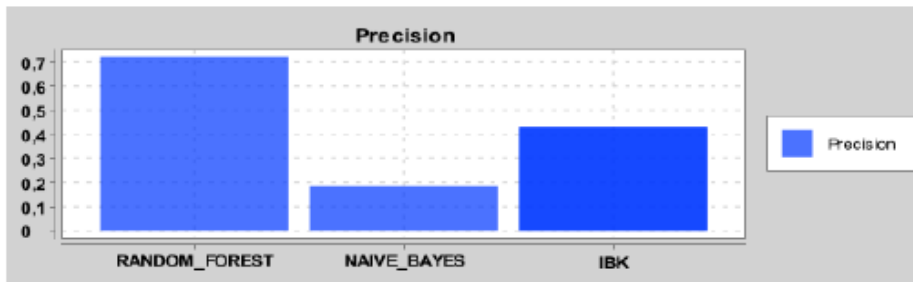
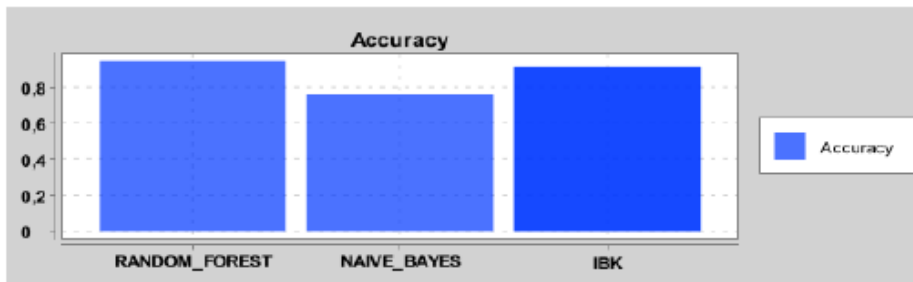
Classifier	TP	FP	TN	FN
RANDOM_FOREST	4559	1393	136367	7231
NAIVE_BAYES	6330	26579	111181	5460
IBK	5139	5962	131798	6651

CUT: NO; FEATURE SELECTION: FILTER; BALANCING: NO; VALIDATION: TTFCV:



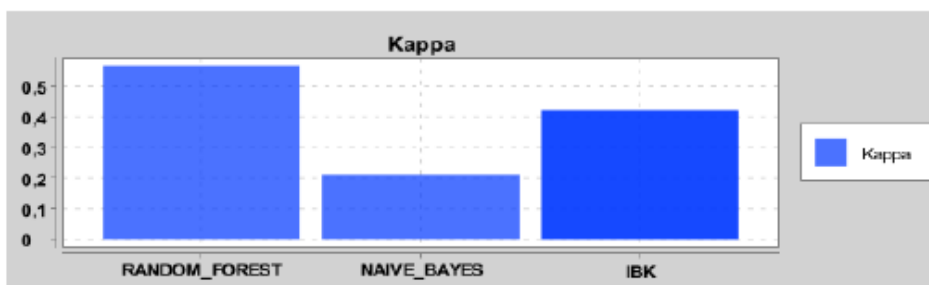
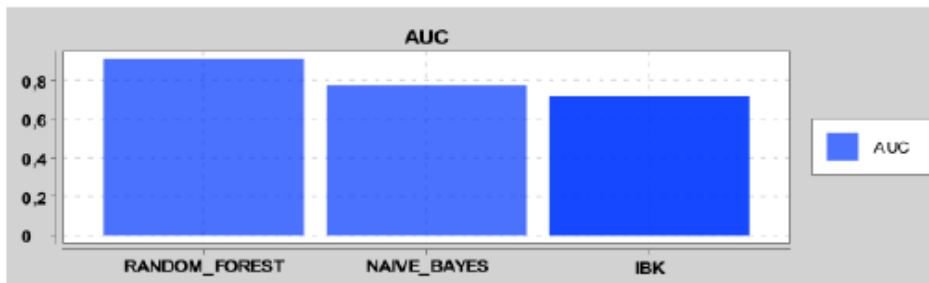
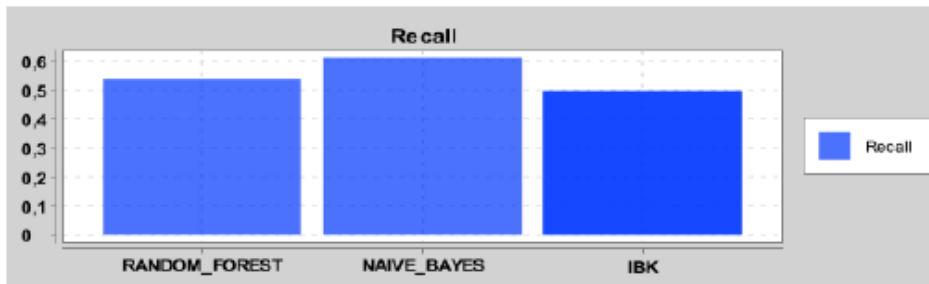
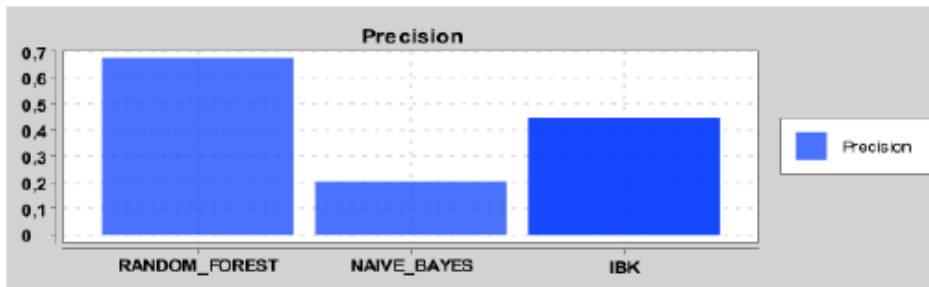
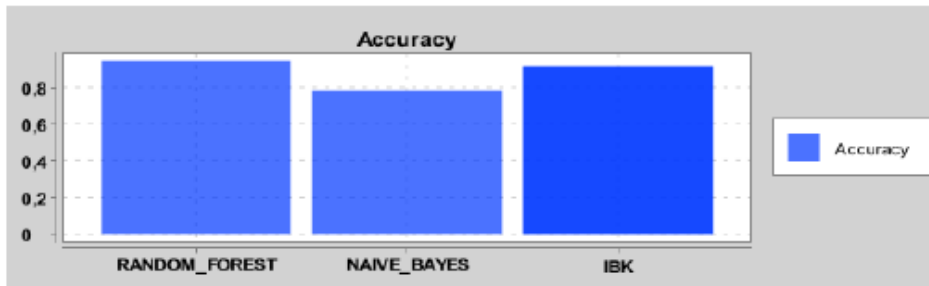
Classifier	TP	FP	TN	FN
RANDOM_FOREST	6129	3558	134202	5661
NAIVE_BAYES	5409	16107	121653	6381
IBK	5698	6704	131056	6092

CUT: YES; FEATURE SELECTION: NO; BALANCING: SMOTE; VALIDATION: TTTFCV:



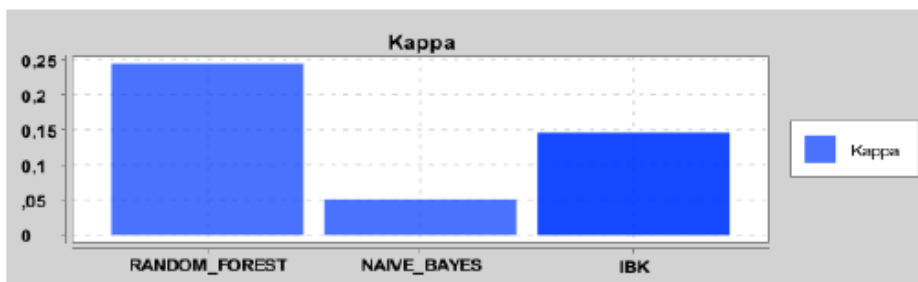
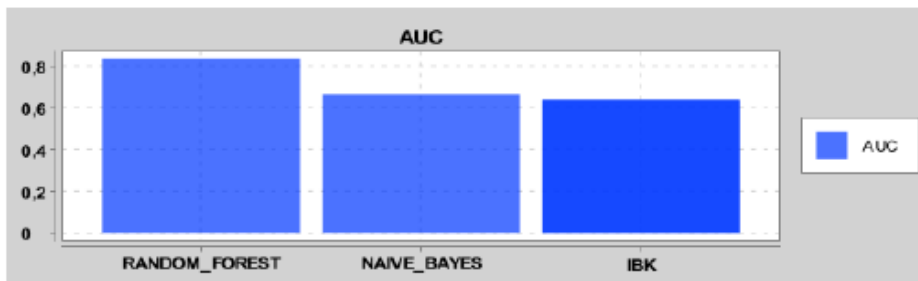
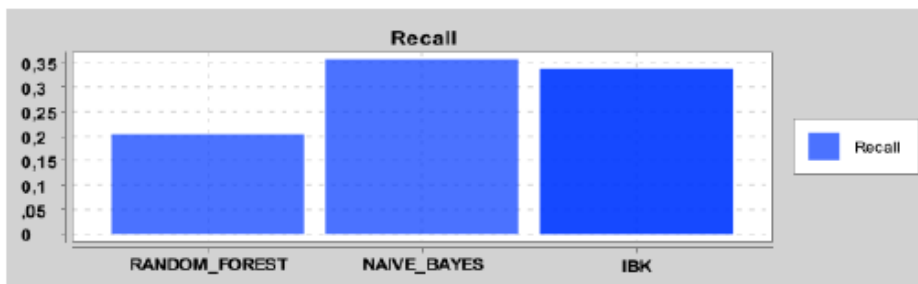
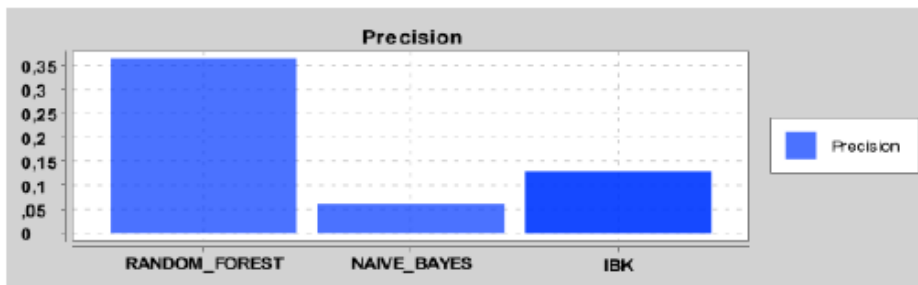
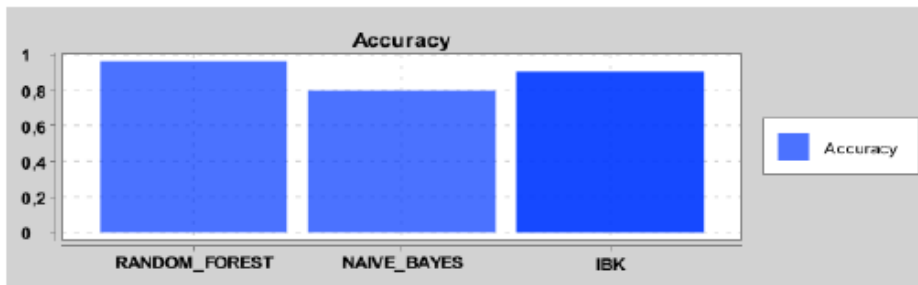
Classifier	TP	FP	TN	FN
RANDOM_FOREST	5480	2123	135637	6310
NAIVE_BAYES	6847	30452	107308	4943
IBK	5505	7257	130503	6285

CUT: NO; FEATURE SELECTION: NO; BALANCING: SMOTE; VALIDATION: TTFCV:



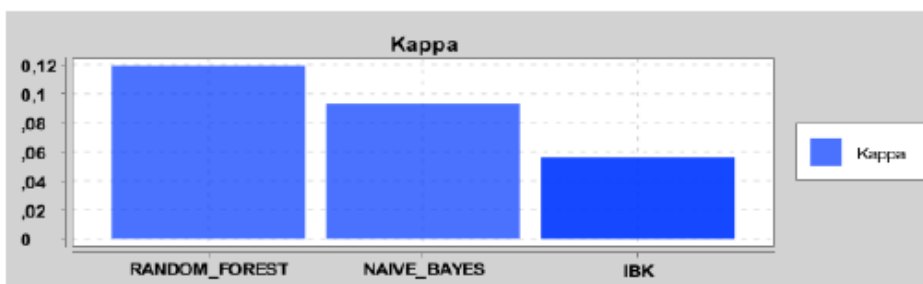
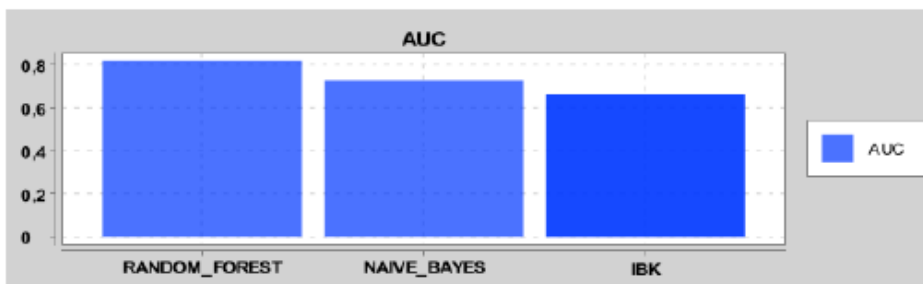
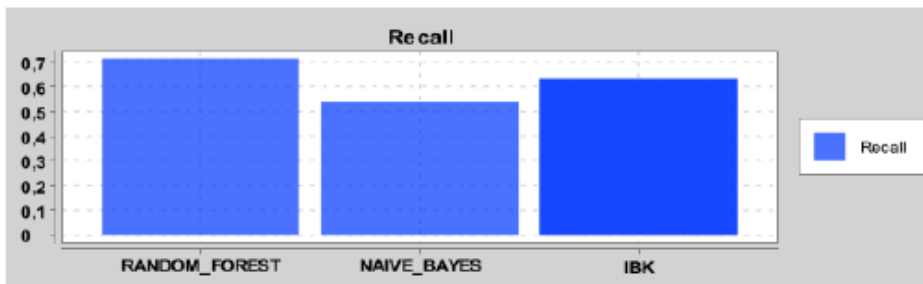
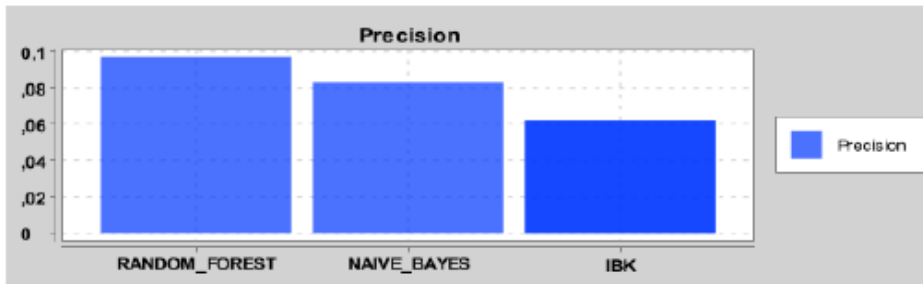
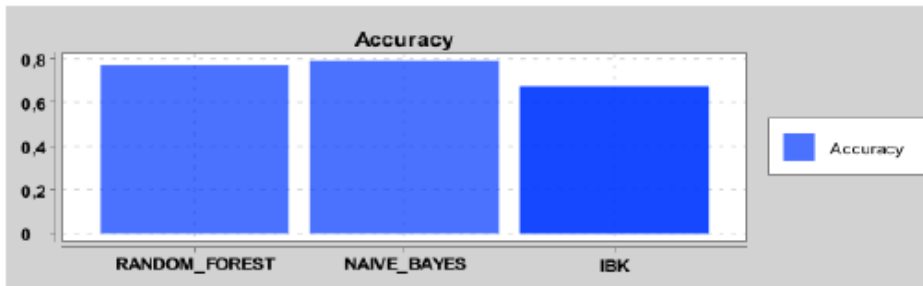
Classifier	TP	FP	TN	FN
RANDOM_FOREST	6305	3038	134722	5485
NAIVE_BAYES	7191	28143	109617	4599
IBK	5831	7187	130573	5959

CUT: YES; FEATURE SELECTION: NO; BALANCING: SMOTE; VALIDATION: OH:



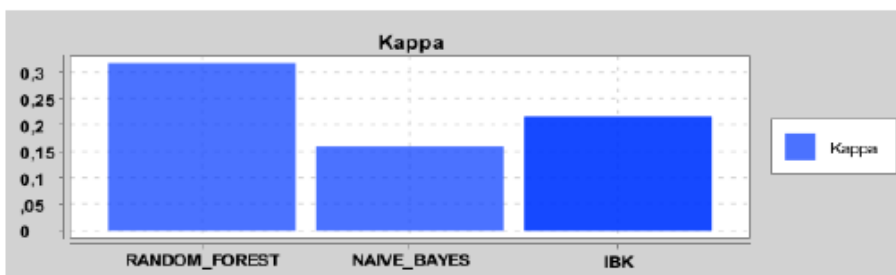
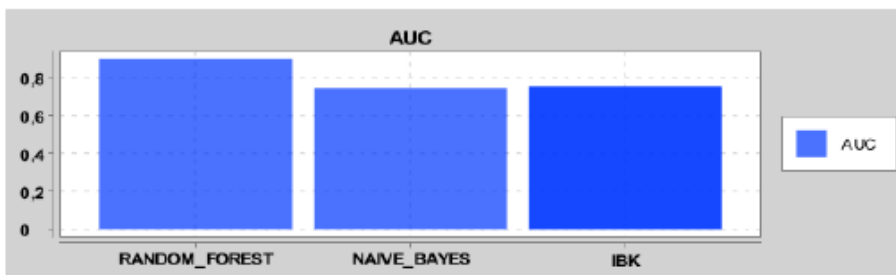
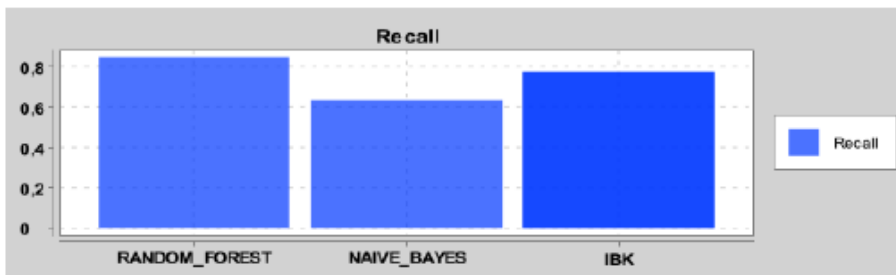
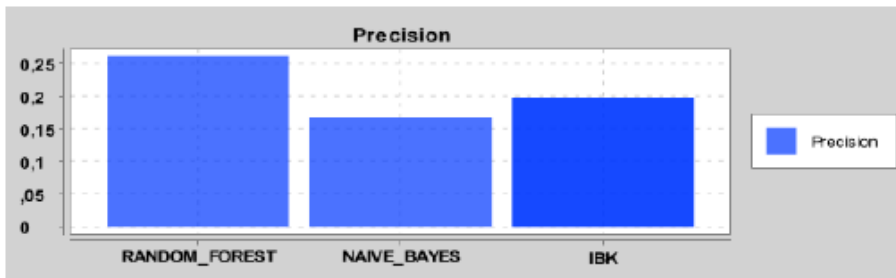
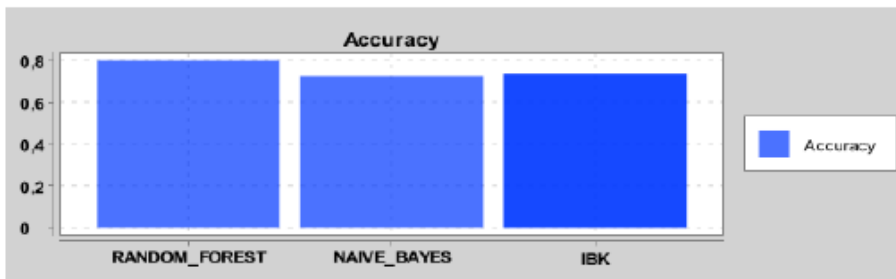
Classifier	TP	FP	TN	FN
RANDOM_FOREST	20	35	2858	78
NAIVE_BAYES	35	540	2353	63
IBK	33	223	2670	65

CUT: YES; FEATURE SELECTION: FILTER; BALANCING: UNDERSAMPLING; VALIDATION: OH:



Classifier	TP	FP	TN	FN
RANDOM_FOREST	70	655	2238	28
NAIVE_BAYES	53	583	2310	45
IBK	62	941	1952	36

CUT: YES; FEATURE SELECTION: FILTER; BALANCING: UNDERSAMPLING; VALIDATION: TTFCV:



Classifier	TP	FP	TN	FN
RANDOM_FOREST	9981	28263	109497	1809
NAIVE_BAYES	7469	37165	100595	4321
IBK	9109	36986	100774	2681